CAD for VLSI

Placement 2

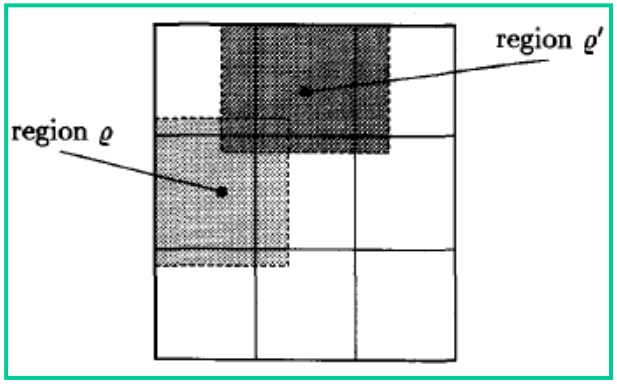
Objectives

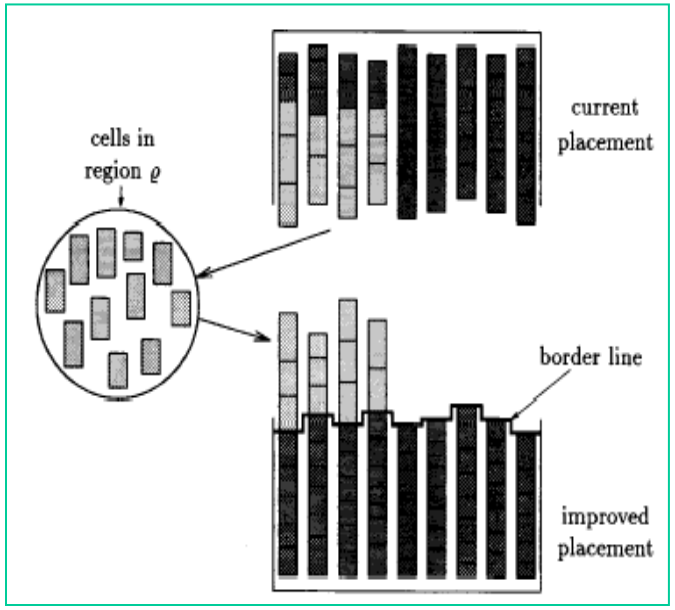
- Major: Legalization
  - Make placement feasible with as little perturbation as possible
- Minor: Further Improvement
  - Wirelength
  - Timing
  - Routability

Representative Approaches

- Constructive methods
  - Network flow [K. Doll et al, TCAD'94]
  - Linear placement [A. Kahng et al, ASPDAC'99]
  - Window interleaving [S. Hur et al, ICCAD'00']
- Iterative method
  - Simulated annealing [W. Sun & C. Sechen, ICCAD'93]
- Hybrid method
  - Network flow + Dynamic programming + Linear programming [J. Vygen, DATE'98] [Brenner et al, ISPD'04]
- Placement migration/spreading
  - Diffusion-based [Ren et al, DAC'05]
  - Delaunay [Luo et al, ICCAD'05]

Constructive Method [K. Doll et al, TCAD'94]





- Overlapped initial placement
- Improve placement
  - Divide chip into overlapping regions
  - Assign cells within each region
  - Iterated if necessary

Constructive Method
[K. Doll et al, TCAD'94]

```
algorithm DOMINO
   initial_placement;
   while (improvement)
      generate_improved_placement;
   end
   adjust_row_lengths;
   swap_cells;
end
```

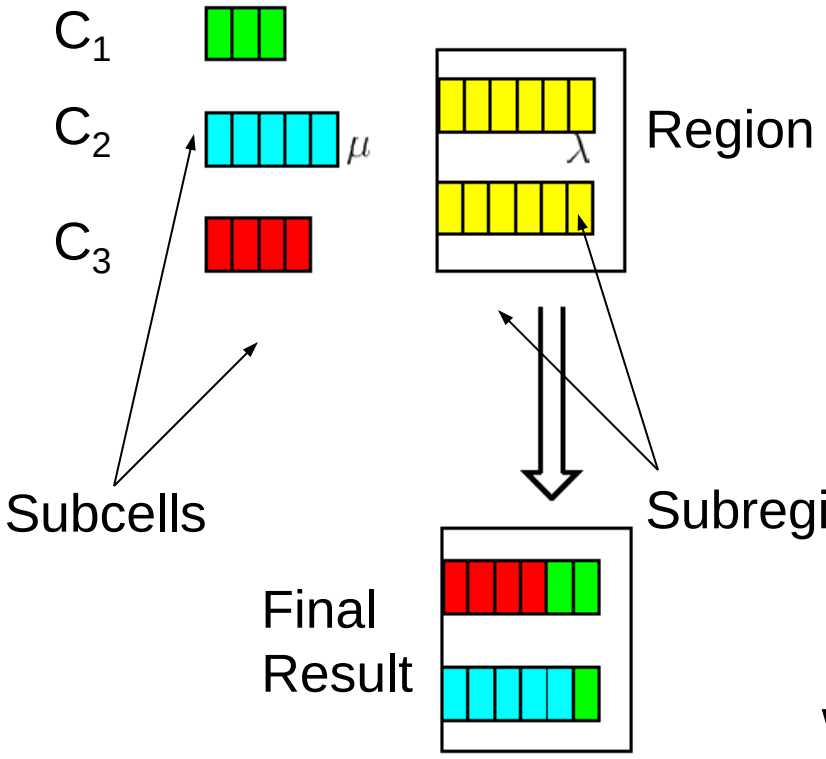
```
algorithm generation
   for all regions ϱ
      get_cells;
      provide_locations;
      solve_transportation_problem;
      move_cells;
   end
end
```

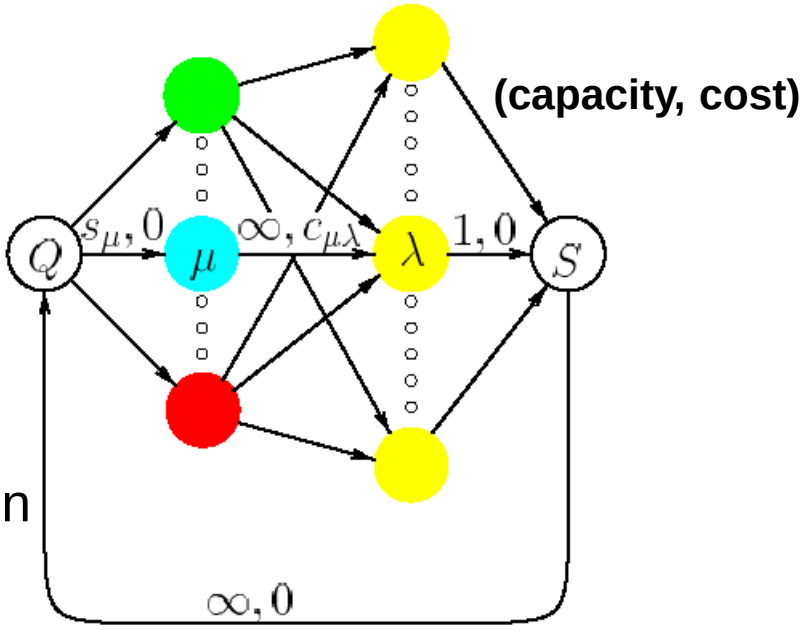
Constructive Method: Network Flow

- Within each region
  - Chop cells into uniform sized subcells
    - All subcells from the same cell have same cost to each candidate location
      - All subcells tend to be pulled towards the cheapest location. As a result, tend to lie side by side
      - In case of separation, move subcells to the column holding the majority
    - Determine y coordinate using gravity center of subcells
  - Transform into min-cost-maximum-flow
    - Introduce source *Q* and sink *S*
    - Capacity from *Q* to any cell μ is s_μ
    - Capacity from candidate location to *S* is 1
    - One edge from each cell to each candidate location with capacity ∞
  - Solve by flow-augmentation algorithm

Constructive Method: Network Flow

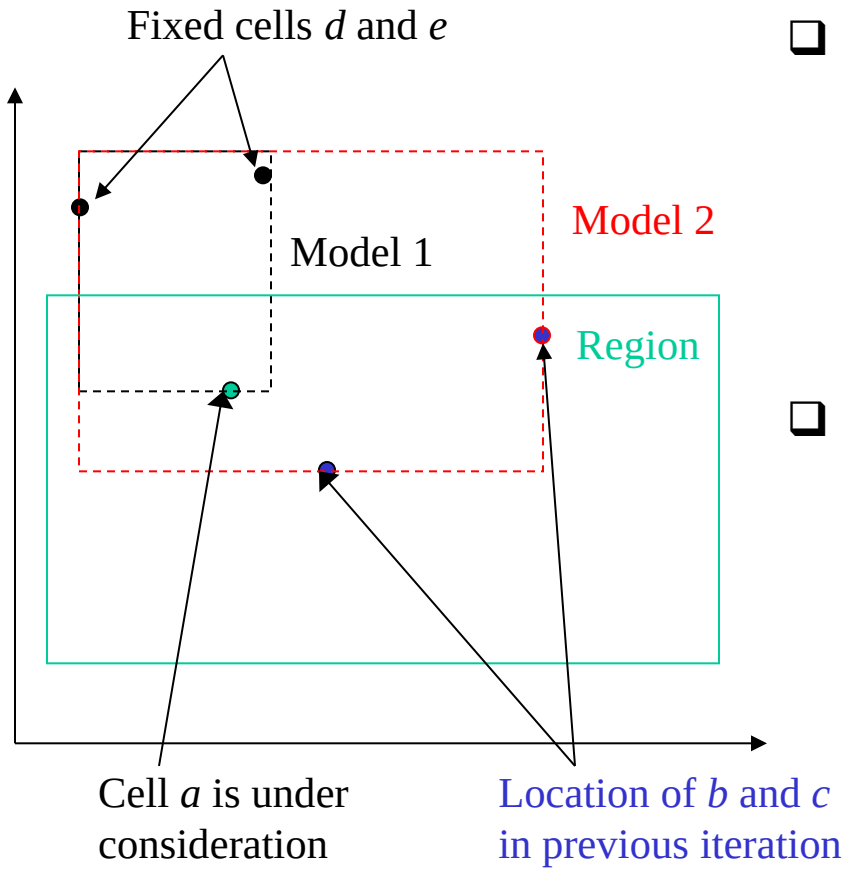
- Network transformation for regions





What about edge costs ?

Constructive Method: Network Flow

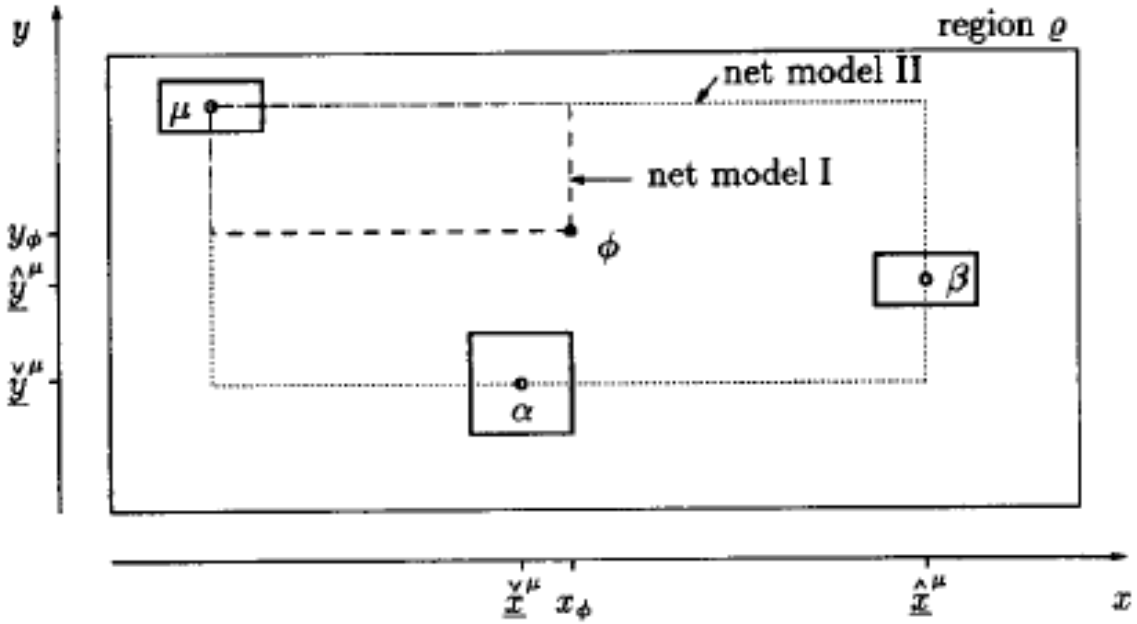


Given a net connecting {*a*,*b*,*c*,*d*,*e*}

- Cost on each edge
  - Depends on wirelength modeling
  - Shall be independent of other internal cells in the region under consideration (because their location is unknown!)
- Two Net models:
  - Model I : Ignore unplaced cells
  - Model II : Use the coordinates from the previous iteration (i.e., the current location) as an approximation
  - Model II consistently better than Model I based on expt results

Constructive Method: Network Flow

- Another scenario of two different net models
- Model 1 introduce a pseudo cell
- Model 2



Constructive Method: Network Flow

circuit	#cell	#pad	#net	#pin	#row
primary1	752	81	904	2941	20
struct	1888	64	1920	5471	19
primary2	2907	107	3029	11229	28
biomed	6417	97	5742	21040	39
industry2	12142	495	13419	48404	66
industry3	15032	68	21924	68290	60
avqsmall	21854	64	22120	60729	80
avqlarge	25114	64	25378	65578	72

Constructive Method: Network Flow

Circuit	TimberWolf V.5.4		VNPR		Gordian-L		Gordian-L + Domino I		Gordian-L + Domino II	
	Area	Runtime	Area	Runtime	Area	Runtime	Area	Runtime	Area	Runtime
primary	20.31	794	23.37	99	20.76	81	19.96	151	20.03	168
struct	6.56	2875	8.23	174	6.48	167	6.38	245	6.2	253
primary2	78.16	5316	95.33	580	78.85	496	78.58	838	78.11	922
biomed	48.22	14631	53.55	5567	45.93	1639	40.43	2742	39.99	2640
industry2	219.52	37521	272.4	16399	233.59	7550	215.61	9835	214.25	9587
industry3	658.01	65652	755.07	11571	598.42	8458	586.92	10257	575.54	10349
avqsmall	161.05	92959	N/A	N/A	129.39	14488	123.57	15963	122.28	17444
avqlarge	168.07	96564	N/A	N/A	141.38	18537	130.1	20549	129.27	21086
average	1	1	1.18	0.21	0.95	0.13	0.9	0.18	0.89	0.18

- Reduces area by 11% on average compared with TimberWolf v.5.4
- Moderate runtime with respect to Gordian-L

Constructive Method [A. Kahng et al, ASPDAC'99]

- Determine the order of cells on a row from global placement
- Fix cells outside the row, and place the cells within the row optimally according to cell order
- Feasible location of cells are discrete, ex: grid points
- Consider white space on the row, i.e., unused space between cells
- Three algorithms are given to exploit different properties of cost function

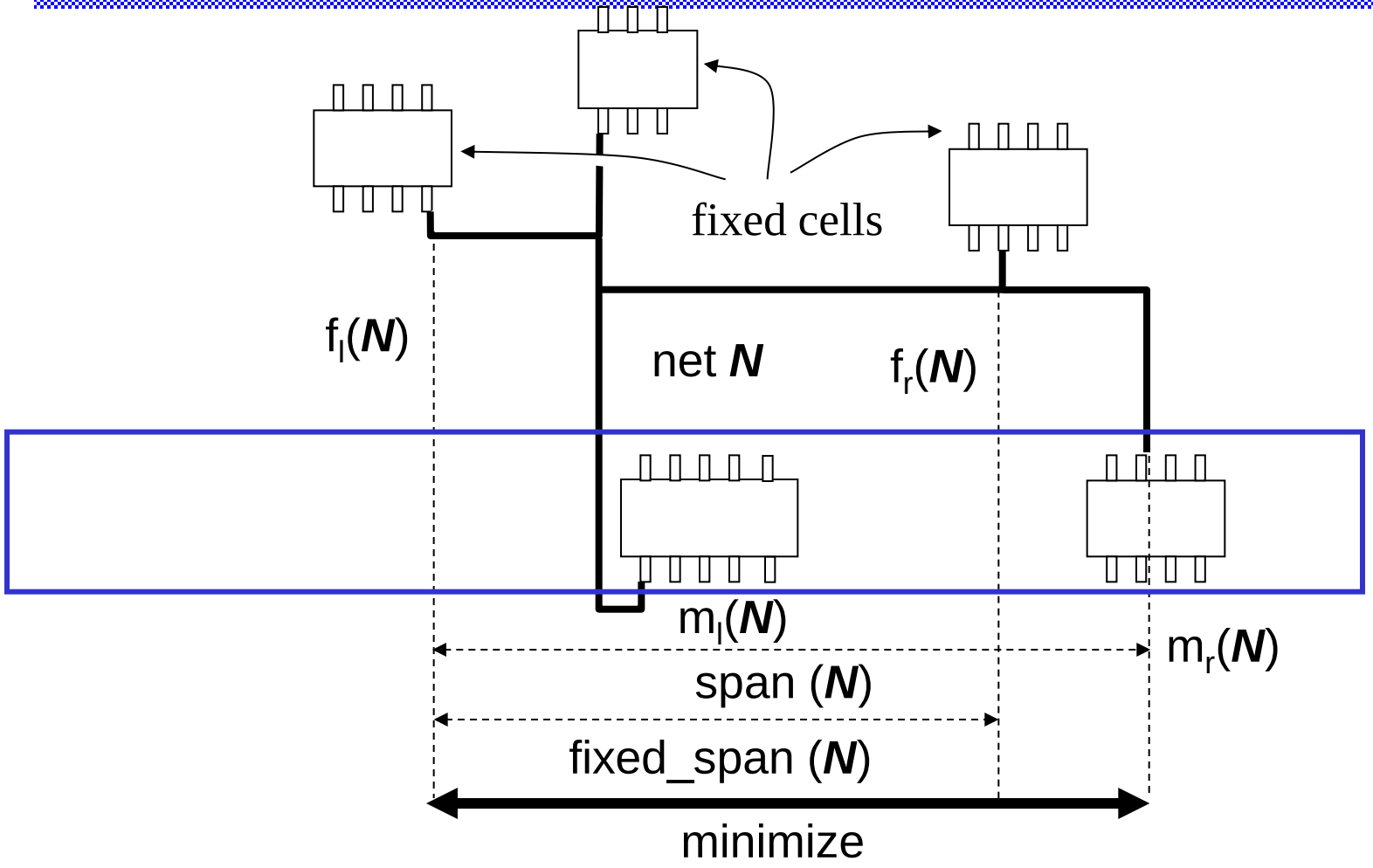
Constructive Method: Linear Placement

- Input
  - A single row with dimension with *n* movable cells c_i, $i=1,2,\ldots m$
  - Fixed cells in other rows
  - Feasible locations S_1, S_2, S_k
  - *m nets,* N_1, N_2, … N_m between the movable and fixed cells
  - Discrete feasible location for cells
- Output
  - Overlapping free placement of c_i within the row with minimum

$$\sum_{i=1}^{n} HPWL(N_i)$$

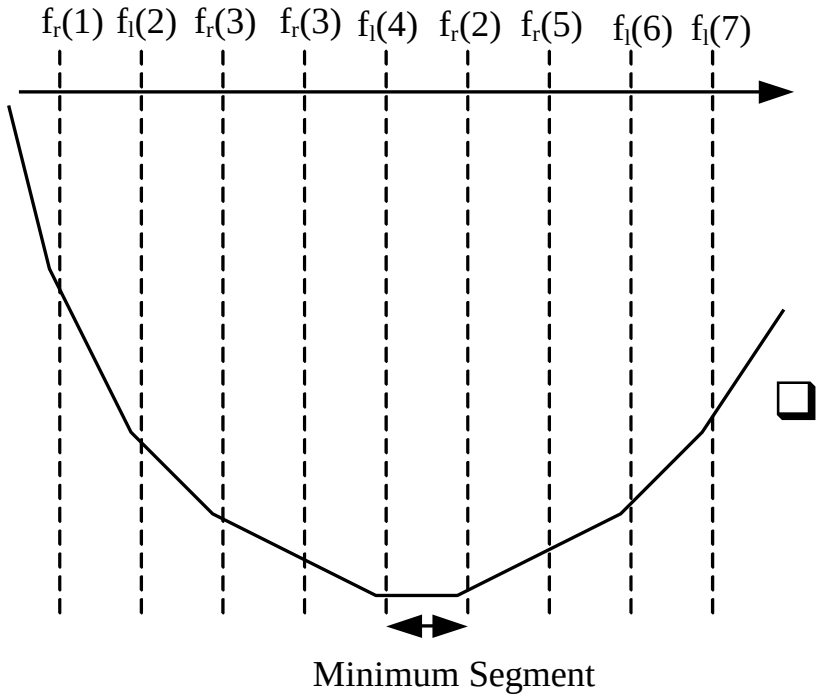
- Constraint (fixed order)
  - $x(c_1) < x(c_2) < \ldots < x(c_n)$

Constructive Method: Linear Placement



Constructive Method: Linear Placement

- Consider contribution of cell c_i



$$cost_i(x) = \sum_{Ci=R(N)} \max\{m_r(N) - f_r(N), 0\} + \sum_{Ci=L(N)} \max\{f_l(N) - m_l(N), 0\}$$

- Piece-wise linear and convex
  - Increasing (decreasing) when $f_r(N)$ to the left of corresponding $l_i(N)$ is less (more) than $f_l(N)$ to the right of corresponding $r_i(N)$

Constructive Method: Linear Placement

- Three algorithms
  - Dynamic Programming Algorithm
    - Exploits pre-computed cell cost functions
  - Prefix Algorithm
    - Exploits piecewise-linearity of cell cost function
  - Clumping Algorithm
    - Exploits convexity of cell cost function

Constructive Method: Linear Placement

- Dynamic Programming Algorithm
  - Subproblem $P_i(s_j)$
    - Place cells $c_1 \ldots c_i$, c_i being at or to the left of the ith feasible location s_j,
    - minimize

$$\sum_{k=1}^{i} cost_k(x)$$

  - Recursion
    - w_i is the width of c_i

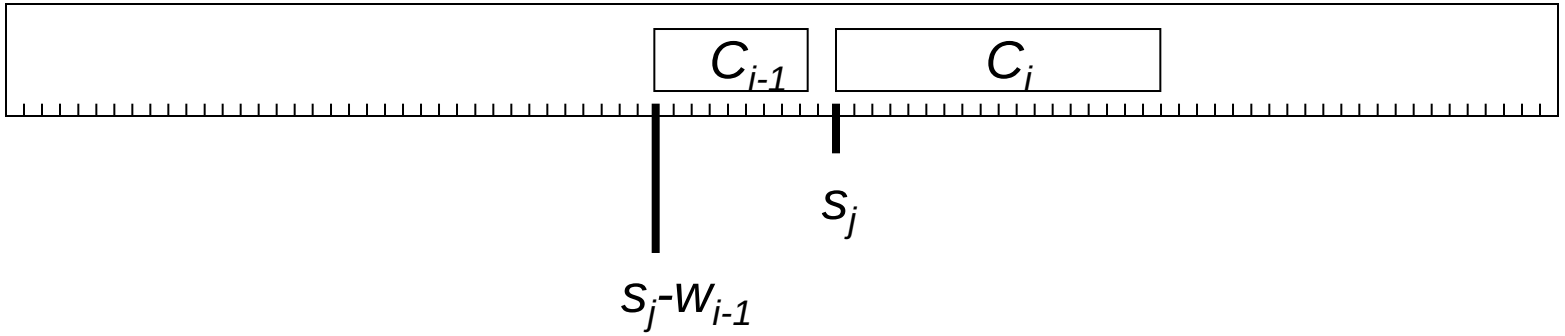
$$P_i(s_j) = \min\{P_i(s_{j-1}), P_{i-1}(s_j - w_{i-1}) + cost_i(x)\}$$

  - Runtime = (i range) $\times$(j range)
    $= n \times (k - \sum w_i)$
    $\approx O(n^2)$

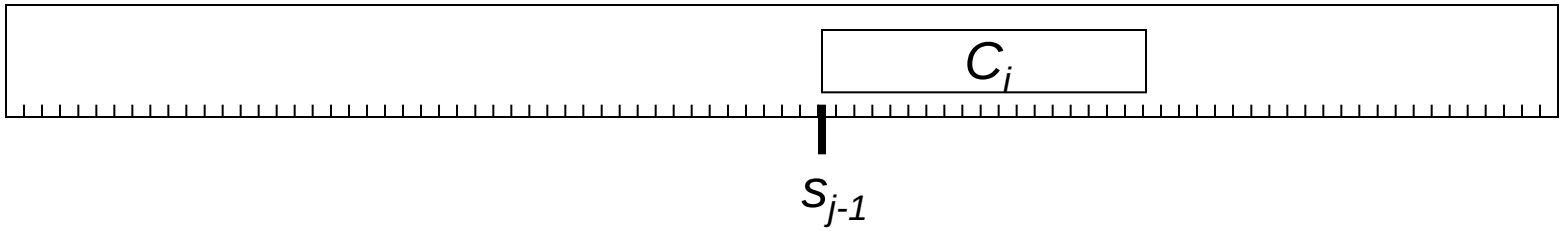
Constructive Method: Linear Placement

$P_i(s_j)$ will be either

c_i exactly at s_j (extend $P_{i-1}(s_j\text{-}w_{i-1})$)



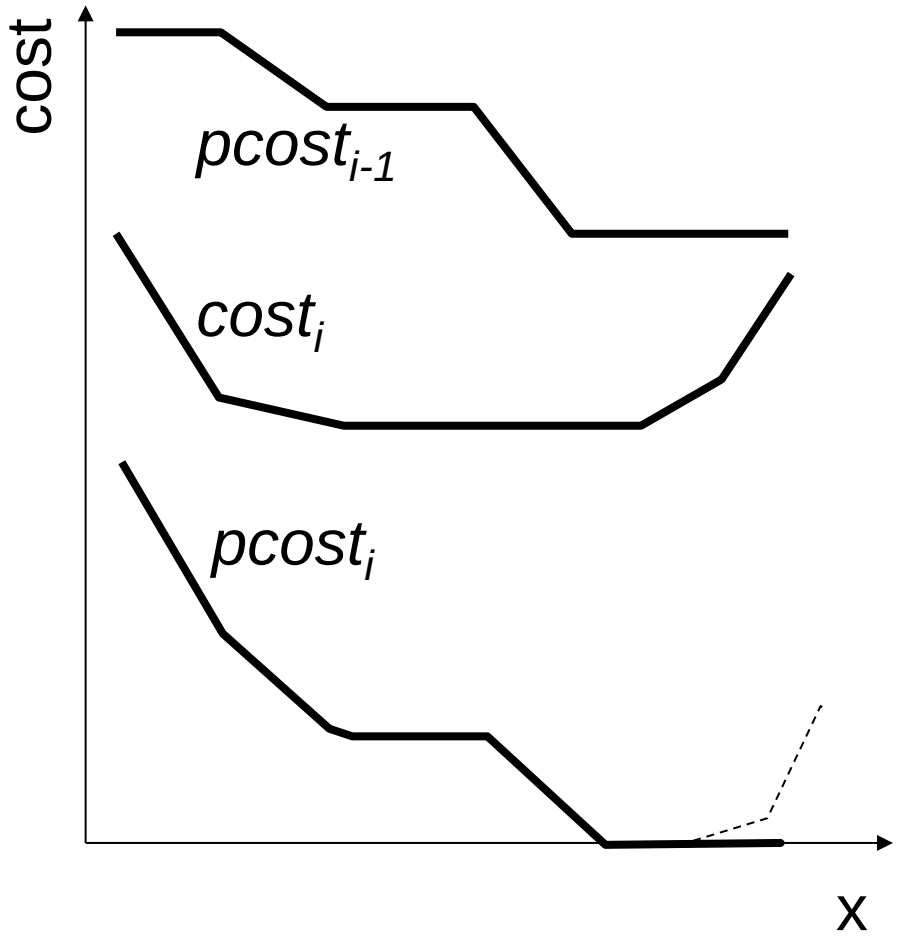
or c_i to the left of s_j (use $P_i(s_{j-1})$)



Constructive Method: Linear Placement

- Exploiting piecewise linearity (PWL) for dynamic programming
  - $pcost_i(x)$ - minimum cost of a prefix placement for the p_i<=x, is monotonically non-increasing, and PWL
    - Why?
  - $pcost_i$ and $cost_i$ can be represented by a sequence of line segments (min, max, slope, y-intercept)
  - Locations bounding maximal line segments of $pcost_i$ is a subset of sites bounding $pcost_i$ and $cost_i$
  - Therefore, consider merging the two representations for each c_i instead of iterate through each feasible location
  - Complexity $O(m^2)$, m is the number nets

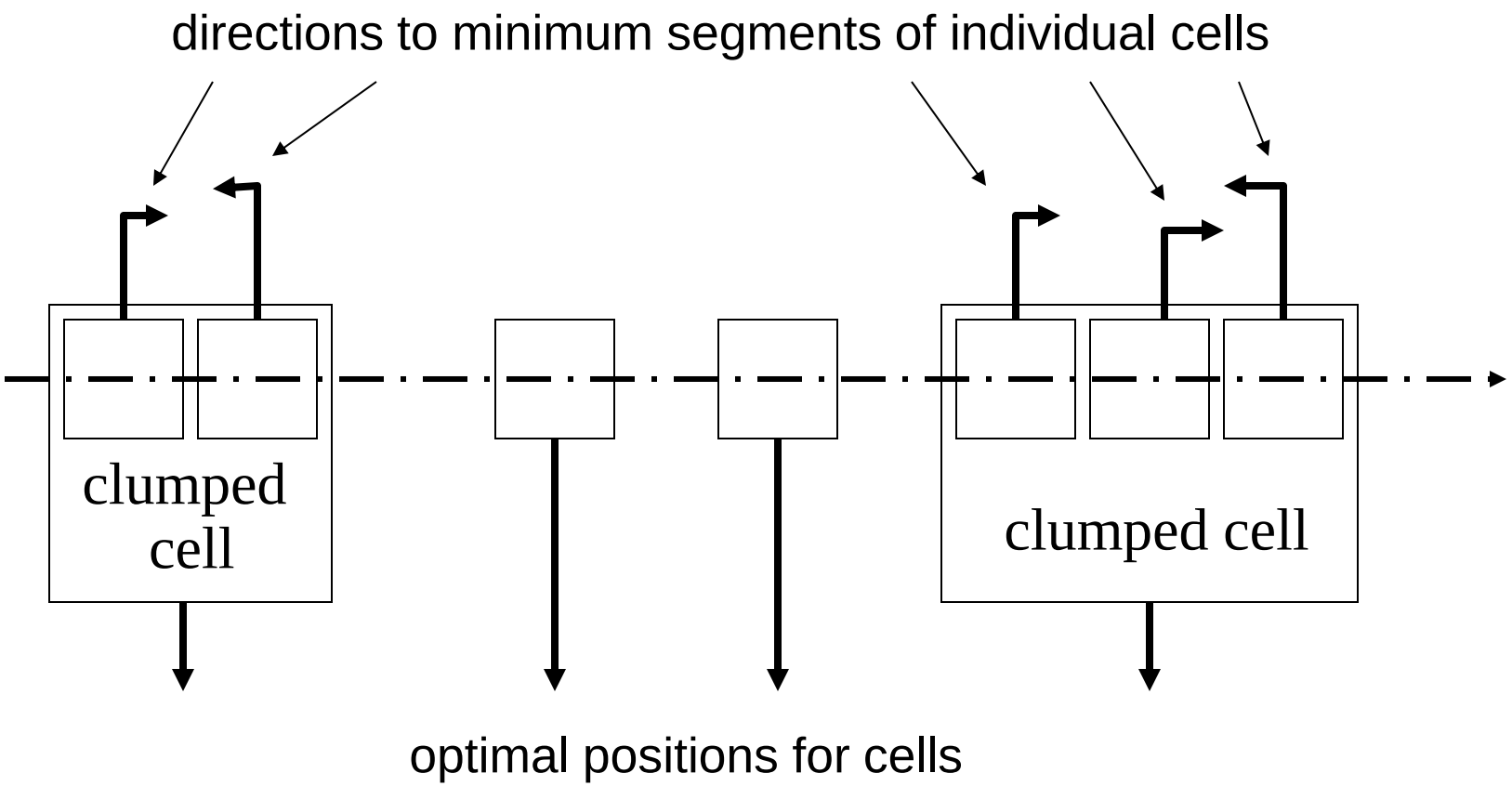
Constructive Method: Linear Placement



Constructive Method: Linear Placement

- Clumping algorithm
- Try to exploit the convexity of the function $cost_i$
  - Find the minimum interval for c_i, i.e., where slope of $cost_i$ is 0
  - Scan adjacent cell starting from left
  - If c_{i-1} and c_i can not be put in their minimum intervals without overlap, replace them with c_{i-1}', with all the pins replicated, then consider placing c_{i-1}'
  - Otherwise place c_i on the leftmost legal site
  - Complexity $O(m^2)$
  - Use RB tree to reduce to $O(m \log m)$

Constructive Method: Linear Placement



Constructive Method: Linear Placement

Circuit			Algorithm	Prefix		Clumping	
#cell	#rows	#site	Heuristic	None	Swap	None	Swap
4155	46	41610	% improvement	9.39	13.38	9.49	13.46
			runtime	56.8	94.1	31.8	69.4
11471	42	78036	% improvement	6	6.18	6	6.22
			runtime	182.9	289	68.9	198
12260	610	128893	% improvement	1.13	1.38	1.15	1.4
			runtime	13.9	35.1	14.8	40.2
7309	56	47152	% improvement	7.43	8.13	7.45	8.08
			runtime	46.5	125.7	35.1	95.9
8829	60	98760	% improvement	3.7	3.54	3.76	3.77
			runtime	34	43.9	20.7	91.8

Comparison with an industry placement tool

Constructive Method [Hur and Lillis, ICCAD'00]

- Start from an overlap-free detailed placement and further improve the wirelength
- Pick an arbitrarily sized interval (window) on a row
- Fixed the cells outside the window, and interleave the cells optimally
- Repeat by sliding the window right across each row

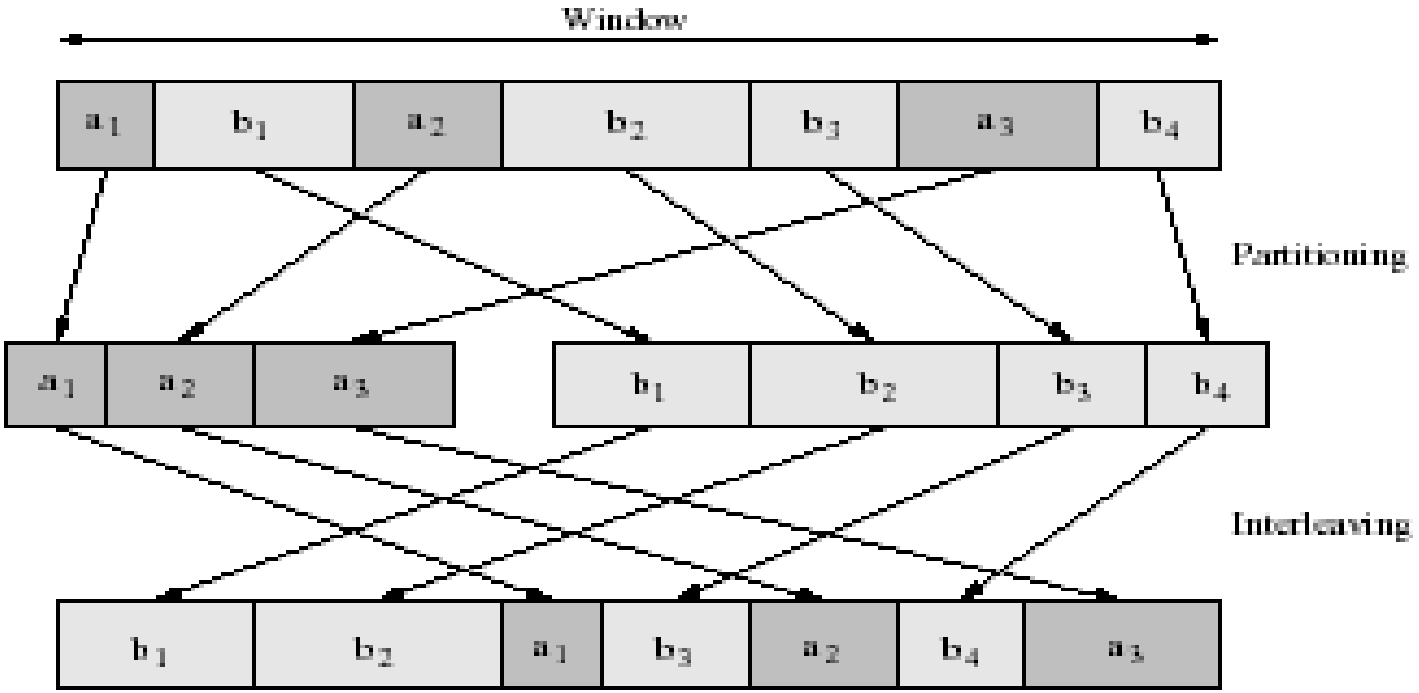
Constructive Method: Window Interleaving

- Input
  - An arbitrarily sized interval *W* within a row
  - Arbitrary ordered subsequence of *W* as $A=\{a_1, a_2, \ldots a_m\}$, and $B = W\text{-}A = \{b_1, b_2, \ldots b_n\}$
  - Fixed cells outside *W*
  - *n nets*, $N_1, N_2, \ldots N_n$ between cells in *W* and fixed cells
- Output
  - Overlapping free placement within *W* with minimum

$$\sum_{i=1}^{n} HPWL(N_i)$$

- Constraint
  - Cell order of A and B is preserved
  - Wirelength of nets connecting cells within *W* is minimized

Constructive Method: Window Interleaving



Ordered sequence *A* and *B* are chosen arbitrarily
The cell order of *A* and *B* is preserved after interleaving

Constructive Method: Window Interleaving

- Dynamic programming
  - Subproblem
    - Denote S_{ij} as the interleaving if $\{a_1, a_2, \ldots a_i\}$ and $\{b_1, b_2, \ldots b_j\}$, with $C(S_{i,j})$ minimized, $C(S_{i,j})$ denotes the cost of S_{ij},
  - Recursion

$$S_{0,0} = 0$$

$$C(S_{0,0}) = 0$$

$$S_{ij} = \begin{Bmatrix} S_{i-1,j}\, a_i, & \text{if } C(S_{i-1,j} a_i) < C(S_{i,j-1} b_j) \\ S_{i,j-1} b_j, & \text{otherwise} \end{Bmatrix}$$

  - Complexity O(nm+p(n+m)), p is the number of pins on incident nets, n and m is the cells in A and B respectively

Constructive Method: Window Interleaving

- Dynamic programming
  - Key point: the optimal interleaving of a prefix $S_{i,j}$ is independent of the ordering of subsequent cells in the window
  - Separability => Dynamic Programming
- Points to ponder
  - Is it optimal?
  - What is the difference compared to the DP in Linear Placement [Kang et al]?
  - If there is no free space, which one will work?

Legalization in Fengshui [ICCAD'03 and ISPD'04]

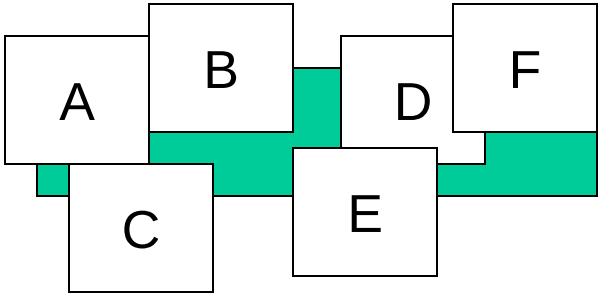
http://vlsicad.cs.binghamton.edu

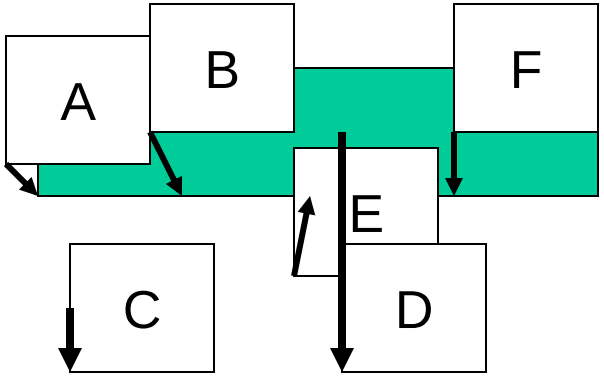
- **Dynamic programming [ICCAD'03]**
- **Greedy Algorithm [ISPD'04]**

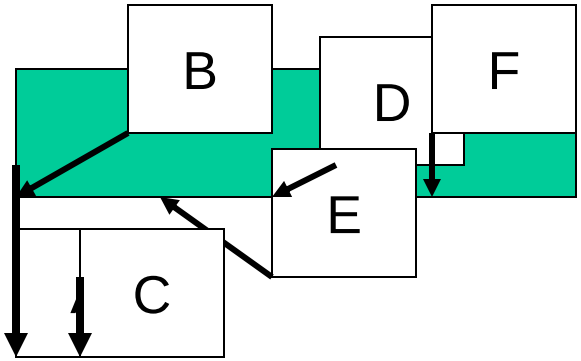
Legalization Method [ICCAD'03]

- Process rows one at a time
- For each row
  - Select a subset of cells such that the total horizontal WL of the packed subset, plus the penalty for the non-selected cells, is minimized
  - Simple dynamic programming
  - Example: six cells, with space for four cells in the row

Example

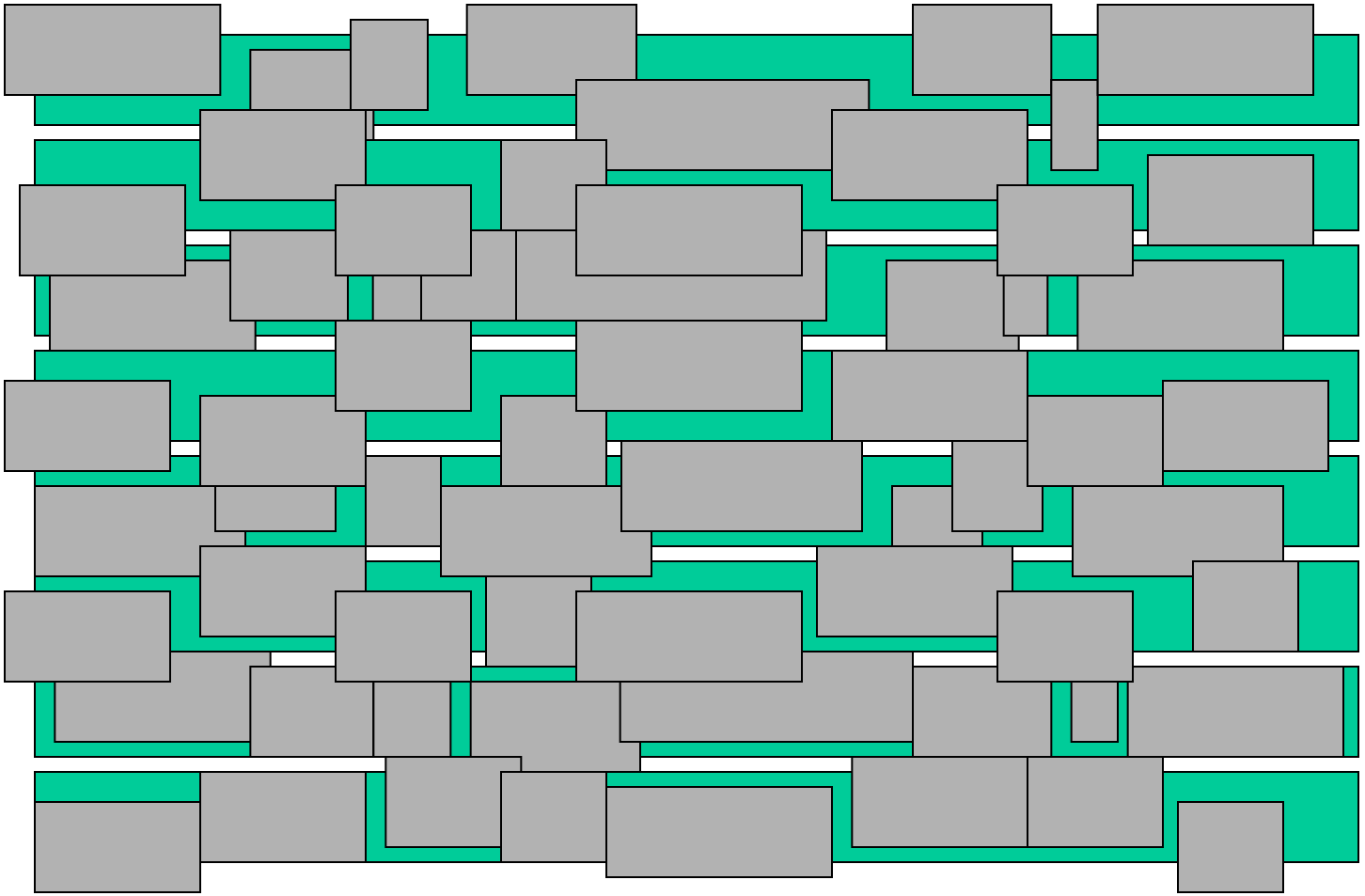




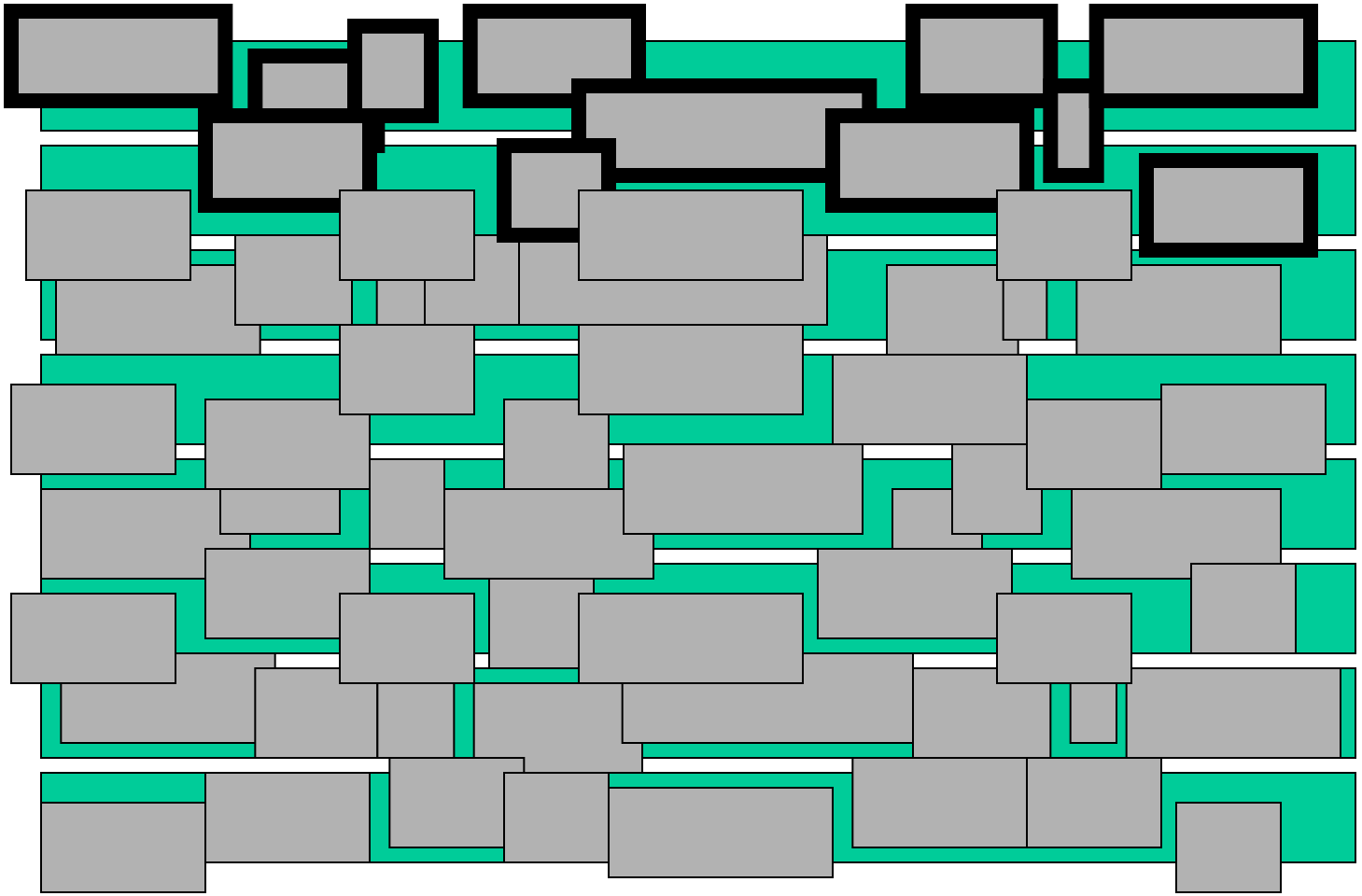


All the blocks are going to be packed to the "left". The total distance things travel depends on which blocks we choose.

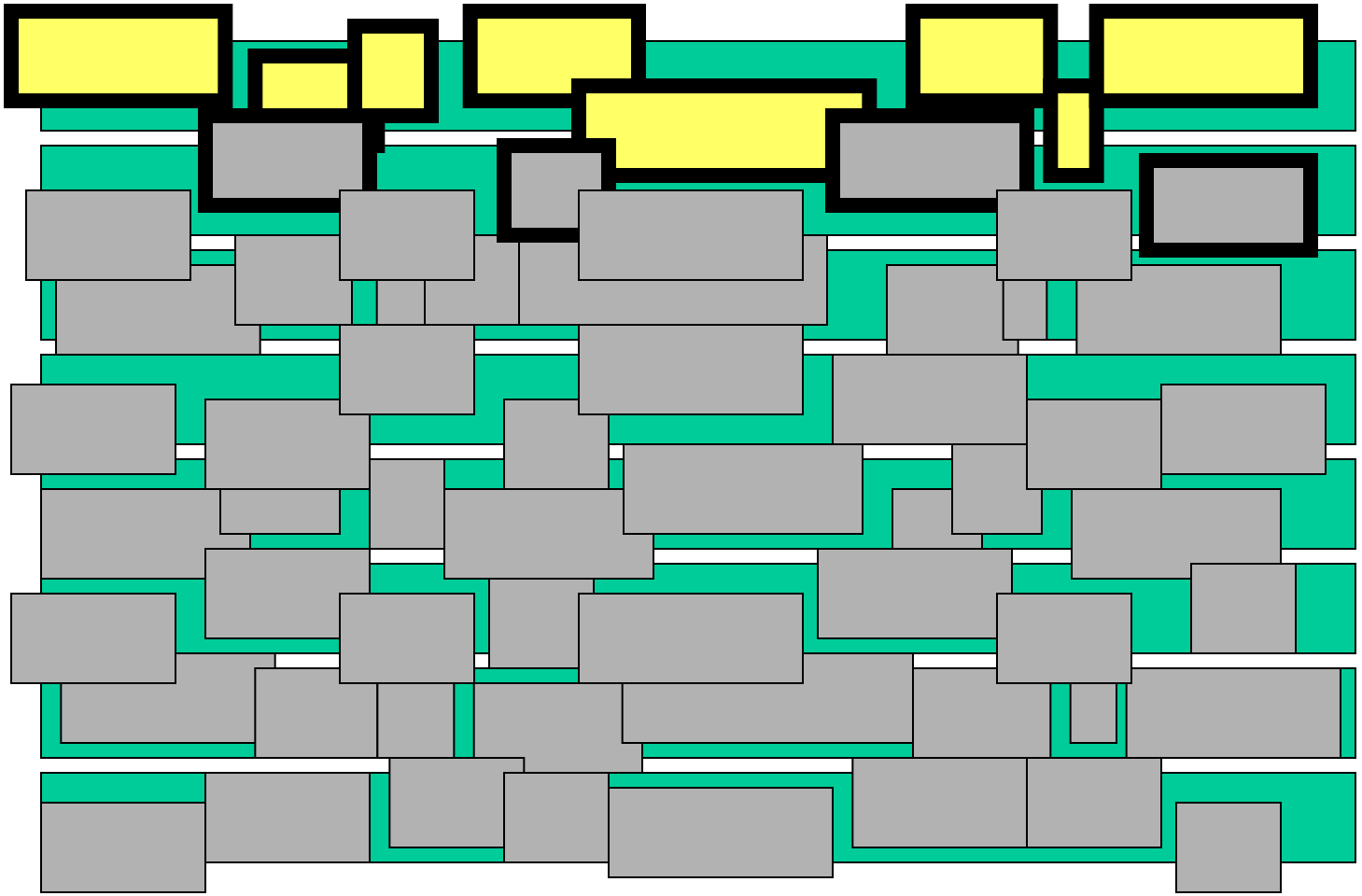
Row by Row



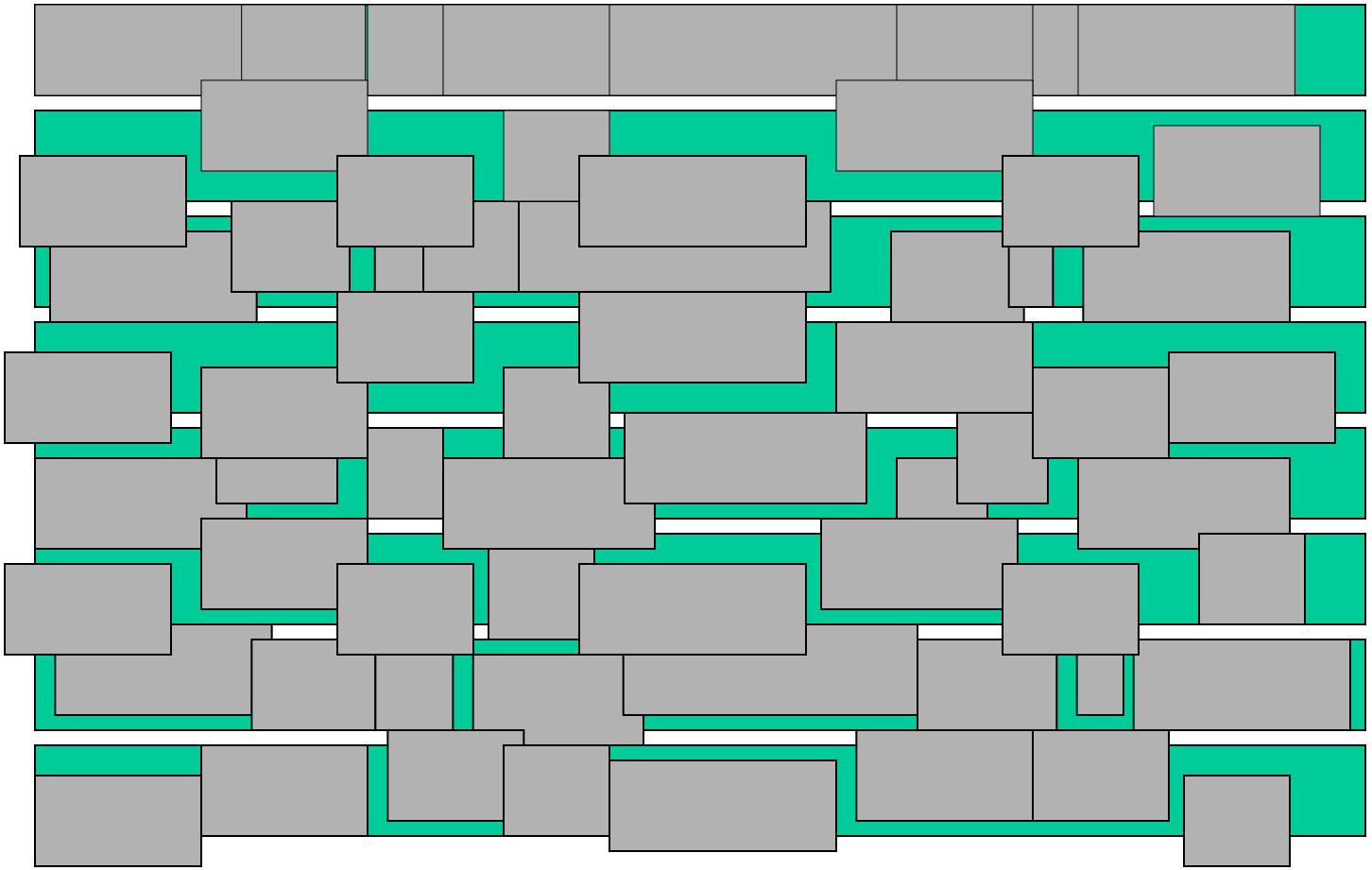
Row by Row



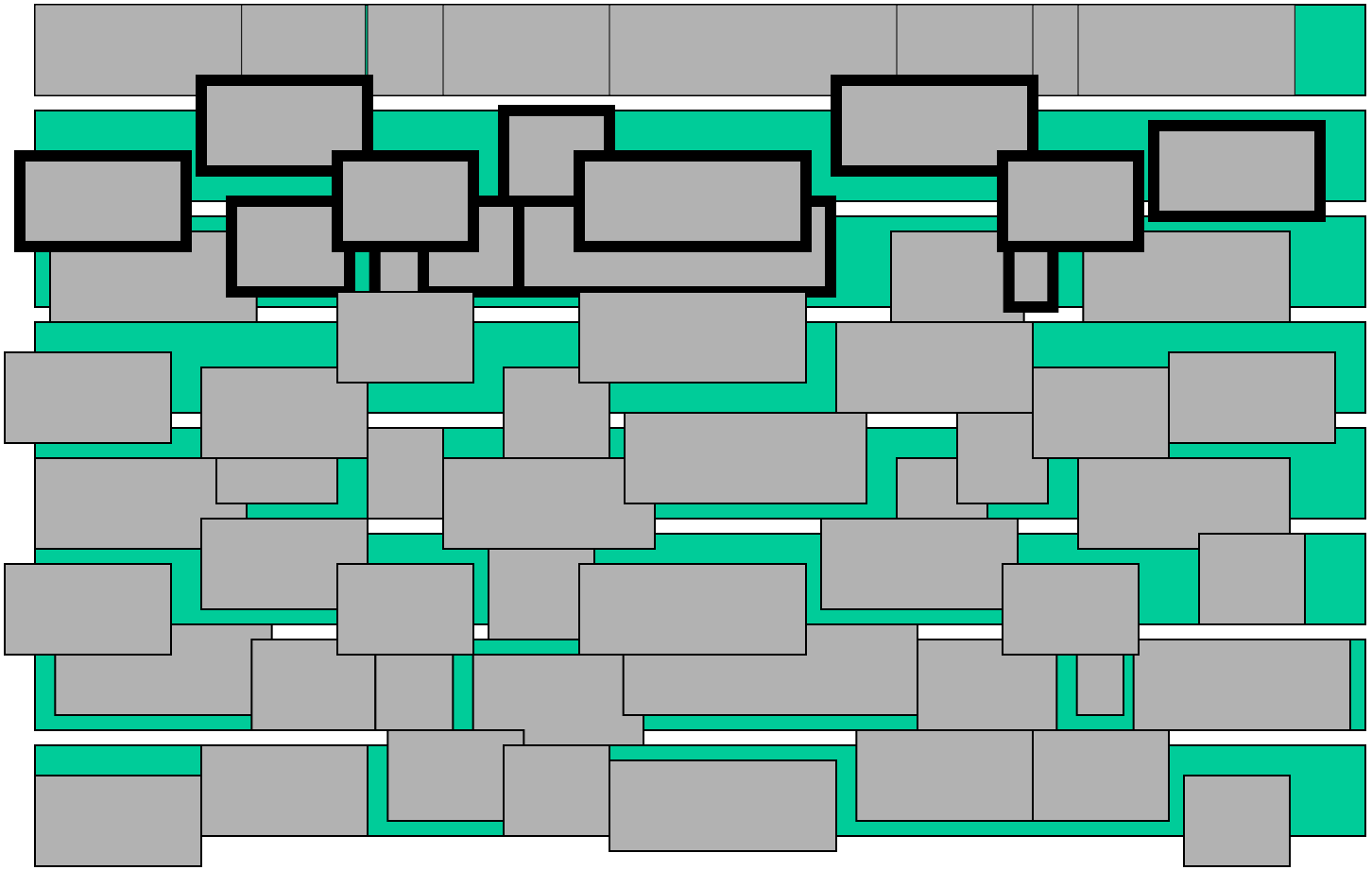
Row by Row



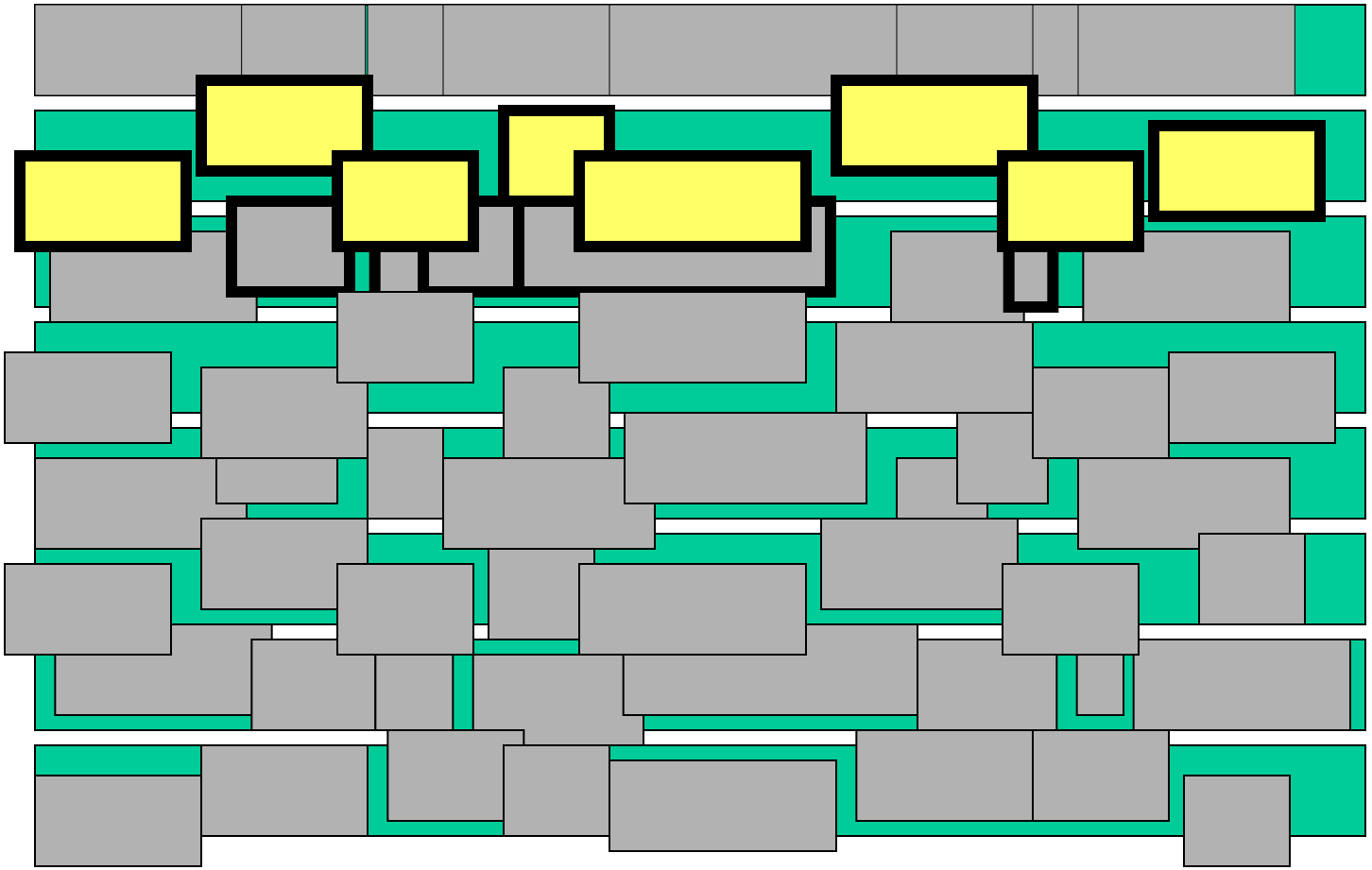
Row by Row



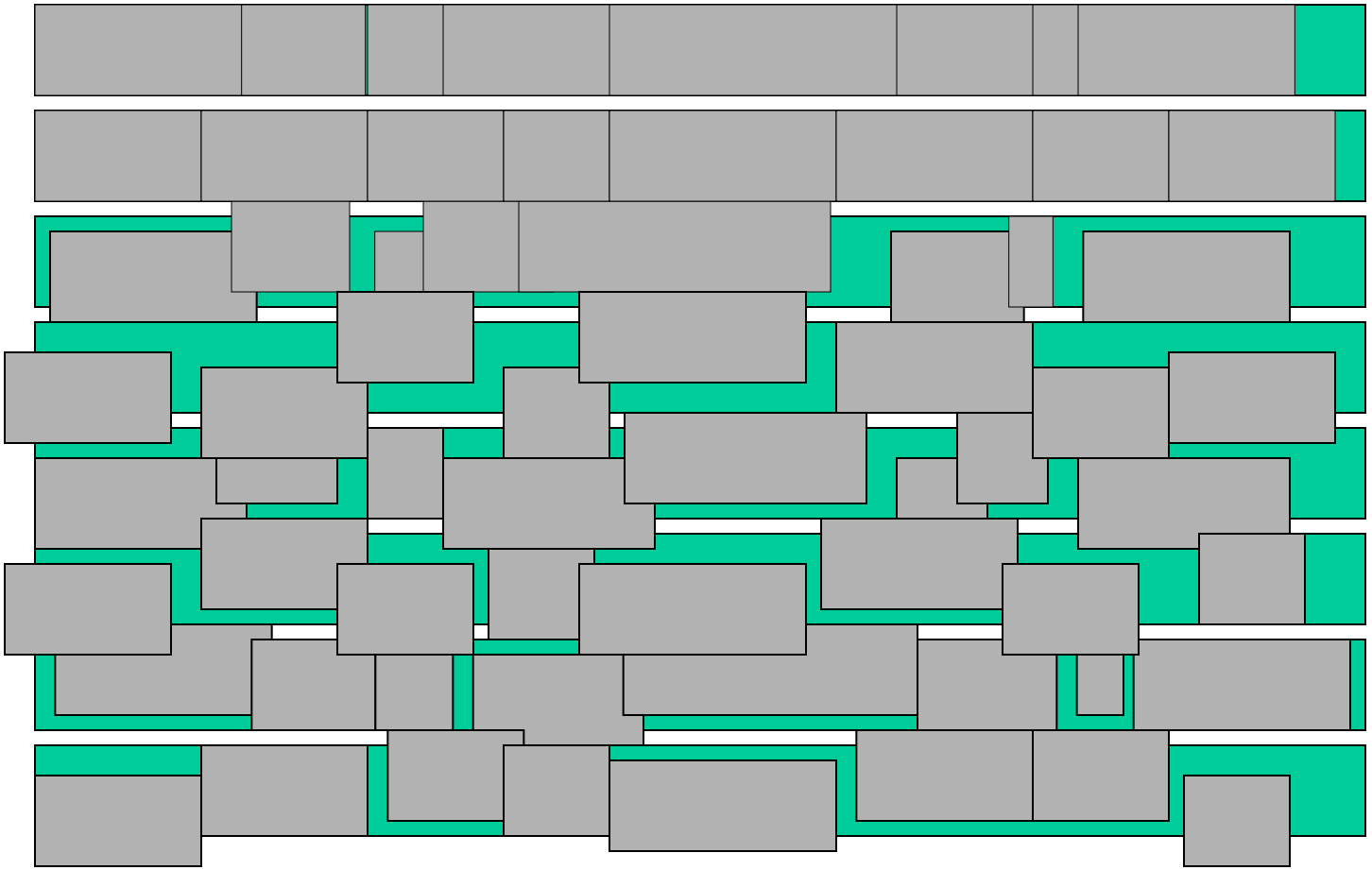
Row by Row



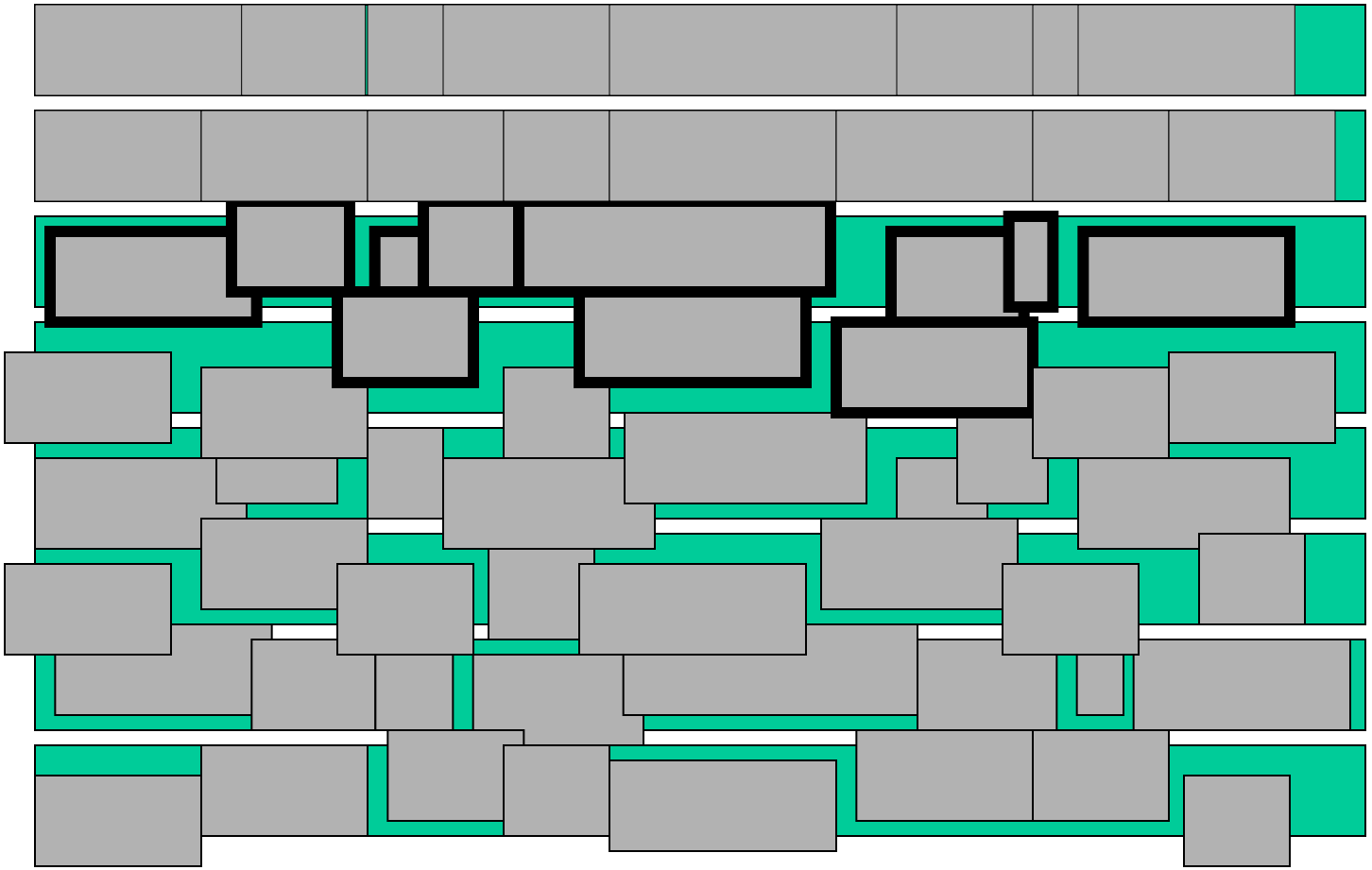
Row by Row



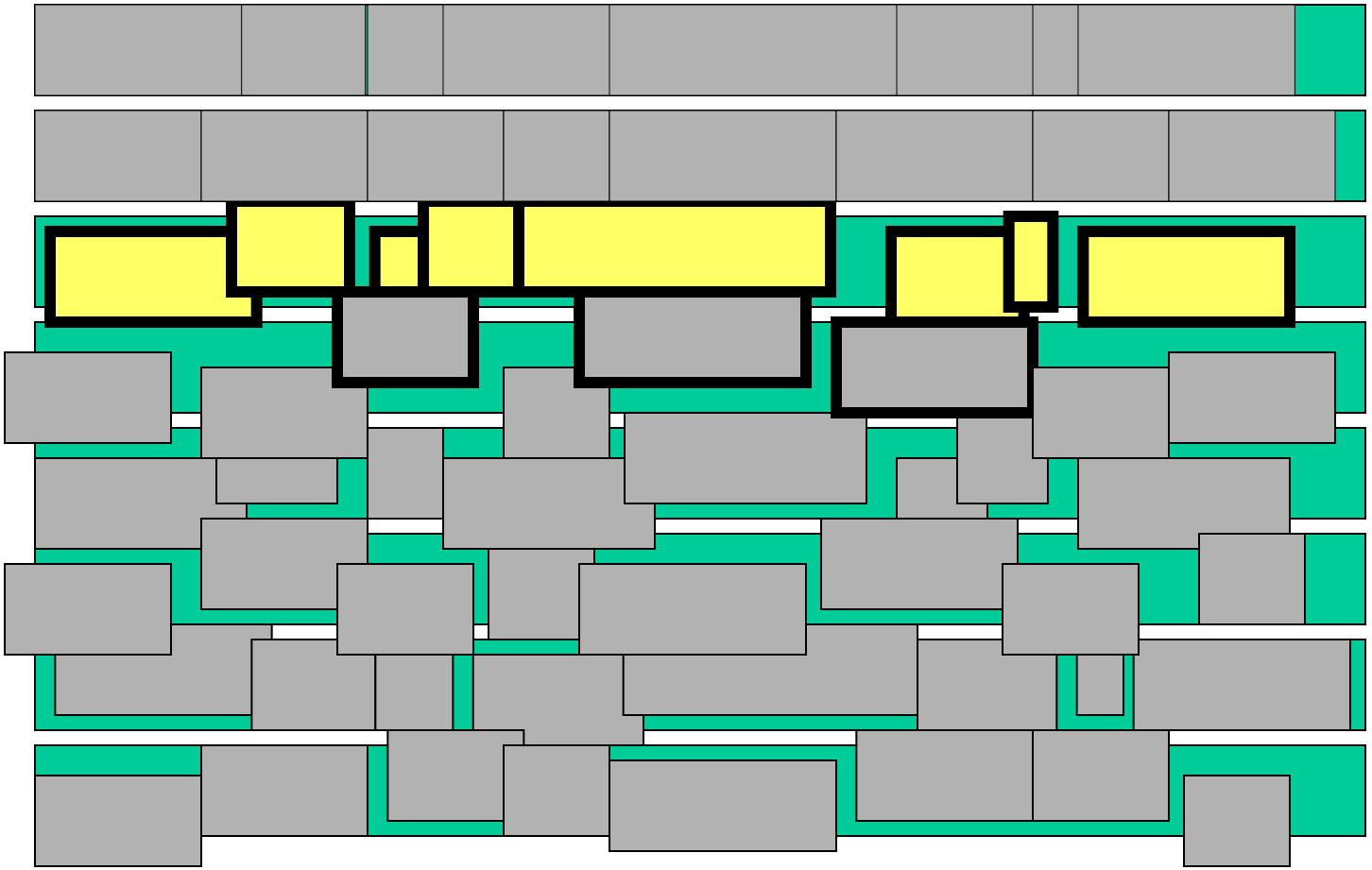
Detailed Placement



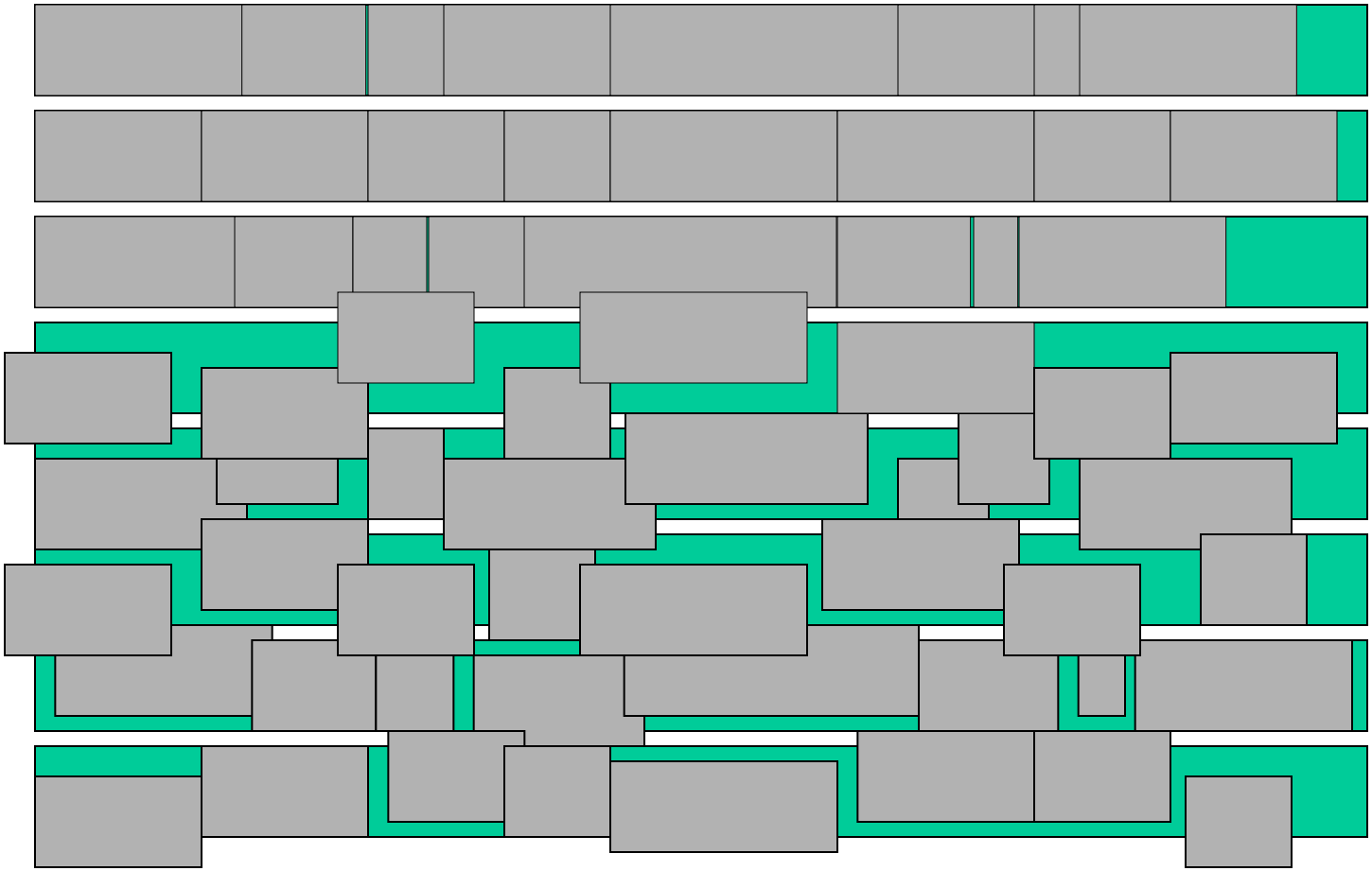
The Third Row



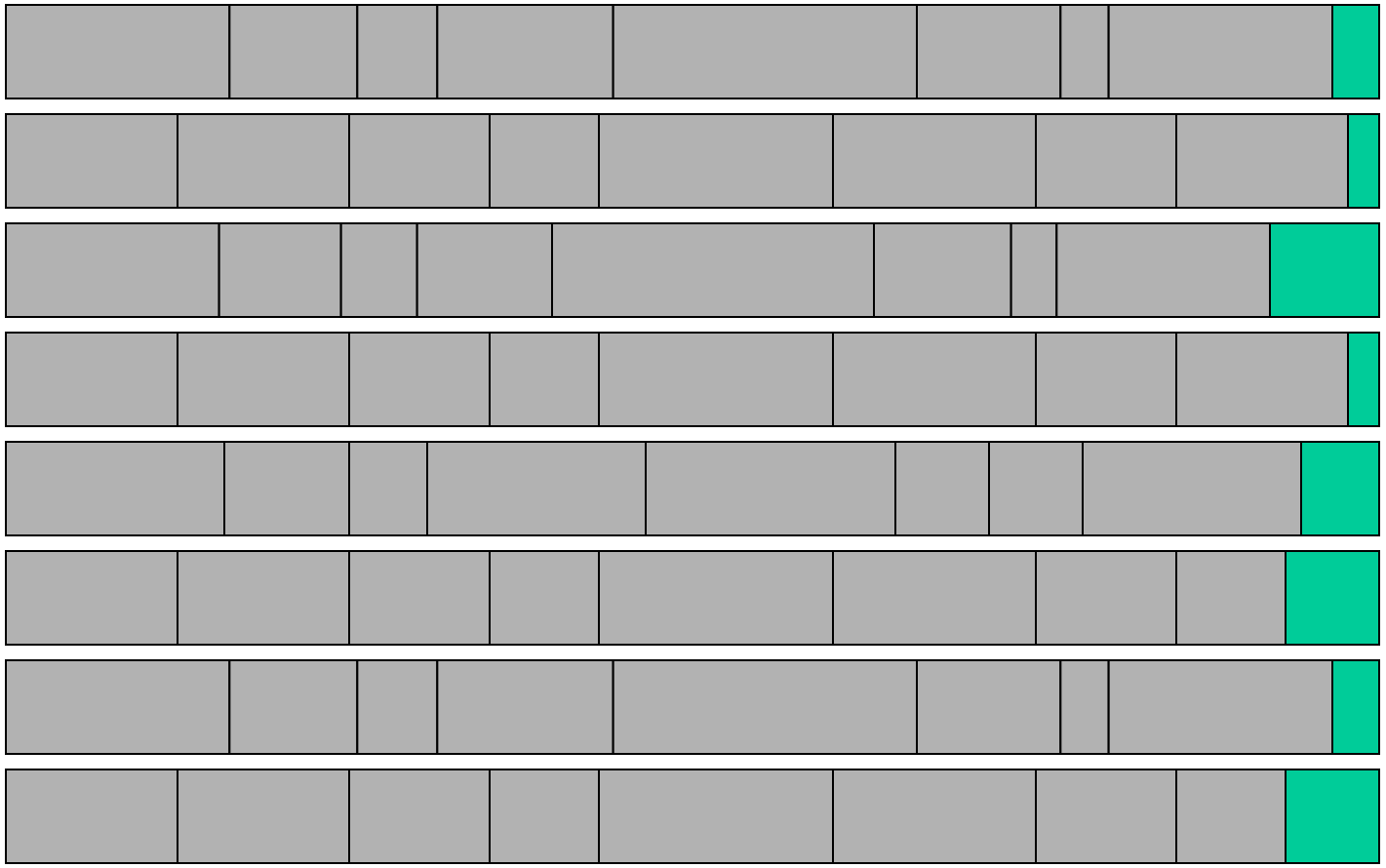
After Bisection



After Bisection



Pack Everything to the Left



Greedy Legalization [ISPD'04]

- Similar to ICCAD'03, but don't solve dynamic programming => Just use greedy methods
- Sort cells/macros by left-edge locations
- Initialize "right edge" of each row
- For each object
  - Greedy assignment of an object to a row, for min displacement
    - If no overlap, leave at abstract placement X position; otherwise shift to the right to avoid overlap
    - Macro placement must check multiple rows
  - Update right edge profiles for the rows across which the object spans
- This method extends a prior standard cell legalization method by Dwight Hill (US Patent 6,370,673)
  - Also used in Kahng/Wang APlace paper [ISPD04]

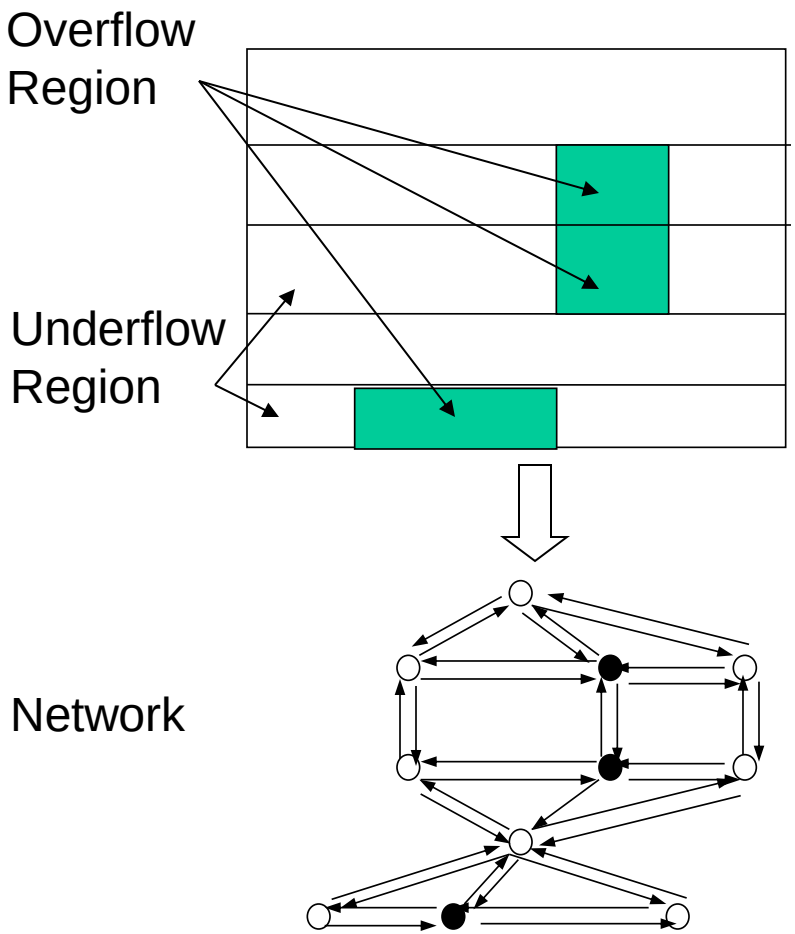
Further improvement by B&B

- After legalization, branch & bound reordering to further improve wire length
- Take a small number of cells from single/multiple rows and try the permutations
- Common “sliding window” technique

Hybrid Method [J. Vygen, DATE'98]

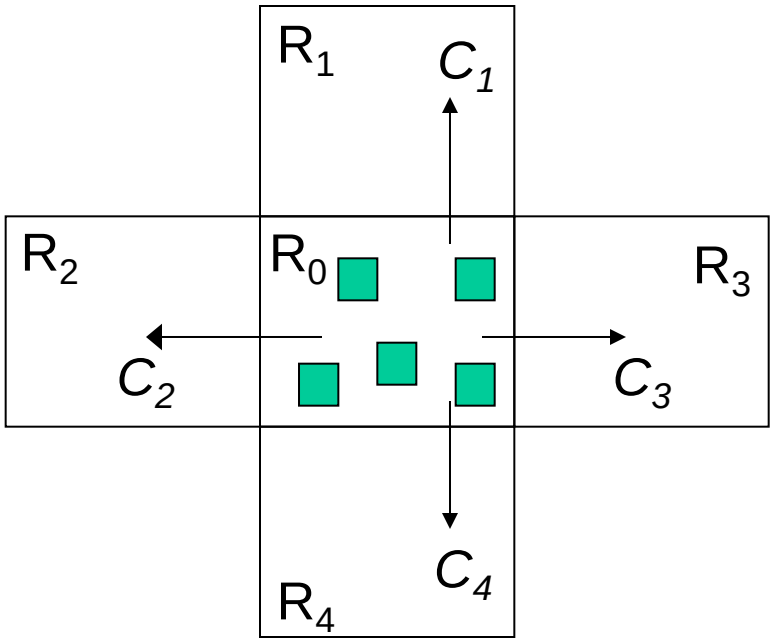
- Legalization among regions
  - Divide the chip into regions using with grid lines and obstacle boundary
  - Determine total amount of cells to move out (into) each region using network flow
  - Decide the cells to move between regions using dynamic programming
- Legalization among zones
  - Divide the rows into zones with no obstacles in them
  - Similar approach to that of regions
- Linear programming within each zone

Hybrid Method: Network flow + Dynamic programming + Linear programming



- Network flow for legalization among regions
  - Regions corresponds to nodes
  - Determine source/sink depending on overflow/underflow
  - Two edges between adjacent regions with uniform cost
  - Result forms a DAG
  - Scan regions in topological order

Hybrid Method: Network flow + Dynamic programming + Linear programming



- Determine the cells to move between regions
  - Need to divide cells in R_0 into $C(r) = \{C_0, C_1, C_2, \ldots C_m\}$, C_i are the cells to move into R_i
  - Subproblem: Partition $\{c_1, c_2, \ldots c_j\}$ into $\{C_0, C_1, C_2, \ldots C_k\}$ with

$$\sum_{i=1}^{k} \sum_{c \in C_k} \text{cost}(c, R, R_k) \quad \text{minimized,}$$

  - cost(c,R, R_i) is the cost moving cell c from R into R_i
  - Solved through dynamic programming

Hybrid Method: Network flow + Dynamic programming + Linear programming

- Linear programming within each zone

$$\min \sum_{n \in N} w(n)(\bar{x}(n) - \underline{x}(n))$$

$$\text{s.t. } \underline{x}(n) \le x(c(p)) + xoffs(p) \le \bar{x}(n) \qquad \text{for each net}$$

$$x^i_{\min} \le x(c^i_1) \qquad \text{left boundary of zone}$$

$$x(c^i_j) + \frac{w(\{c^i_j, c^i_{j+1}\})}{2} \le x(c^i_{j+1}) \qquad \text{nonoverlap}$$

$$x(c^i_{m_i}) \le x^i_{\max} \qquad \text{right boundary of zone}$$

- Theoretically sound, but no experimental results given

Hybrid Method [Brenner et al, ISPD'04]

- Follow up work on Vygen's 1998 work
- With experimental results
- Please see their slides on ISPD'2004 web site
  - http://www.ispd.cc

Placement Migration

- From an initial placement (either legal or illegal), migrate to another placement solution
- It happens all the time during physical synthesis for multi-objective design closure

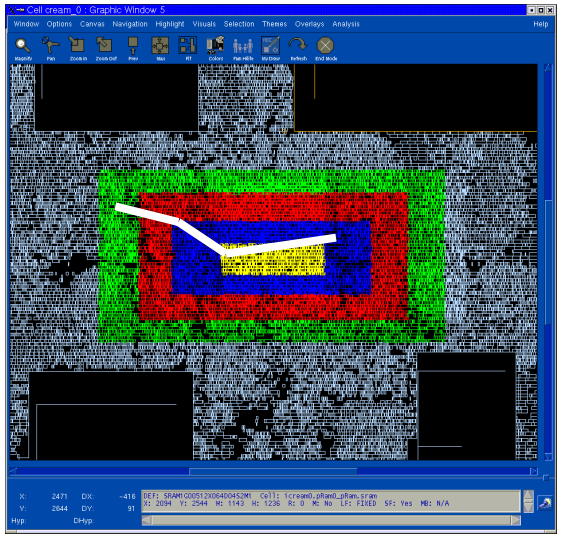
1. Post/detailed placement
   - Legalization due to buffer insertion, gate sizing, Engineering Change Order (ECO), or decoupling capacitor insertion.
   - Congestion/noise mitigation
   - Thermal hot spots removal
   - ...
2. Global placement
   - Cell spreading is a KEY step in any analytical engine

Prior Work

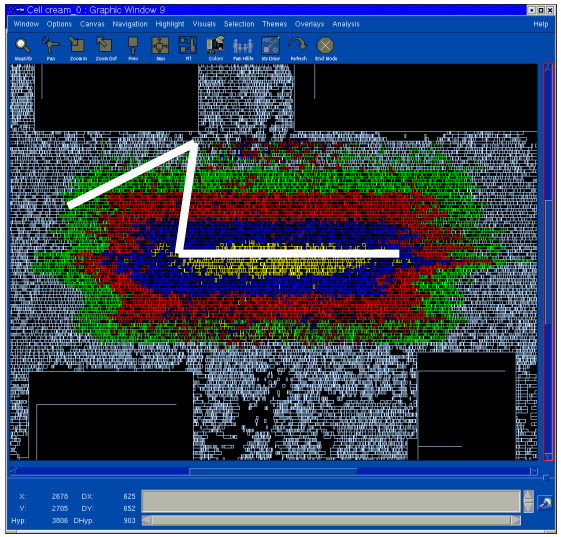
- There are many previous works in legalization/spreading
  - Greedy approach [Viswanathan & Chu, ISPD'04]]
  - Force-directed [Eisenmann & Johannes, DAC'98]
  - Network flow [Brenner et al, ISPD'04]
  - Dynamic Programming [Kahng et al, ASPDAC'99; Hur & Lillis ICCAD'00]
  - Global re-placement from a late cut [Ren et al, ICCAD'04]
  - …

However, none of them attempted to PRESERVE the relative geometry order in a continuous manner – important for nanometer design closure!

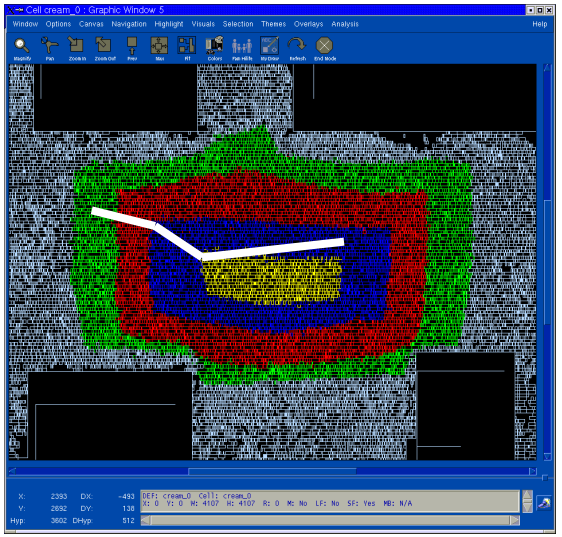
Diffusion Based Placement Migration [Ren et al, DAC'05]



Initial placement (illegal)



Legal placement from greedy alg.



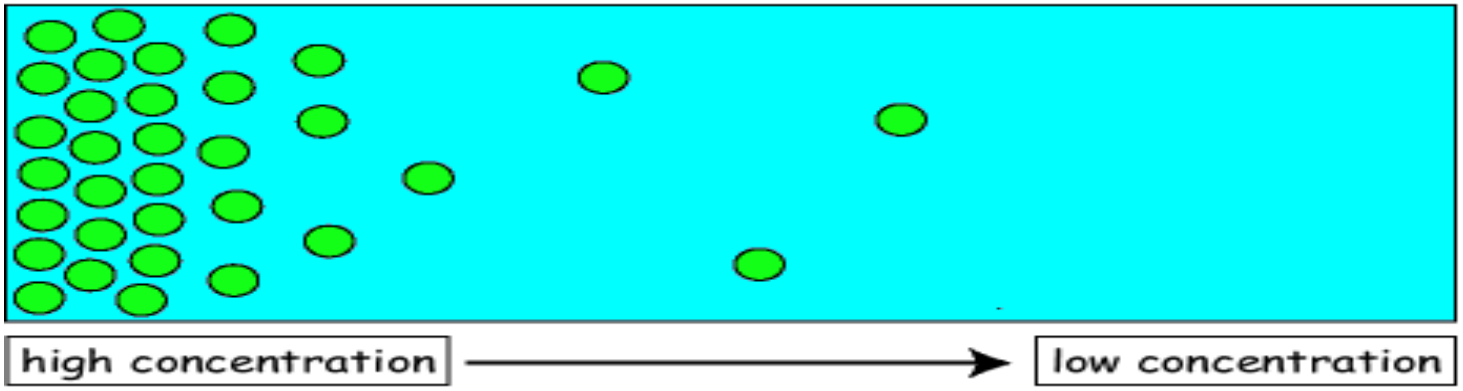
Legal placement from our Diffusion Based Placement [Ren+, DAC'05]

What is Diffusion?

- Diffusion equation

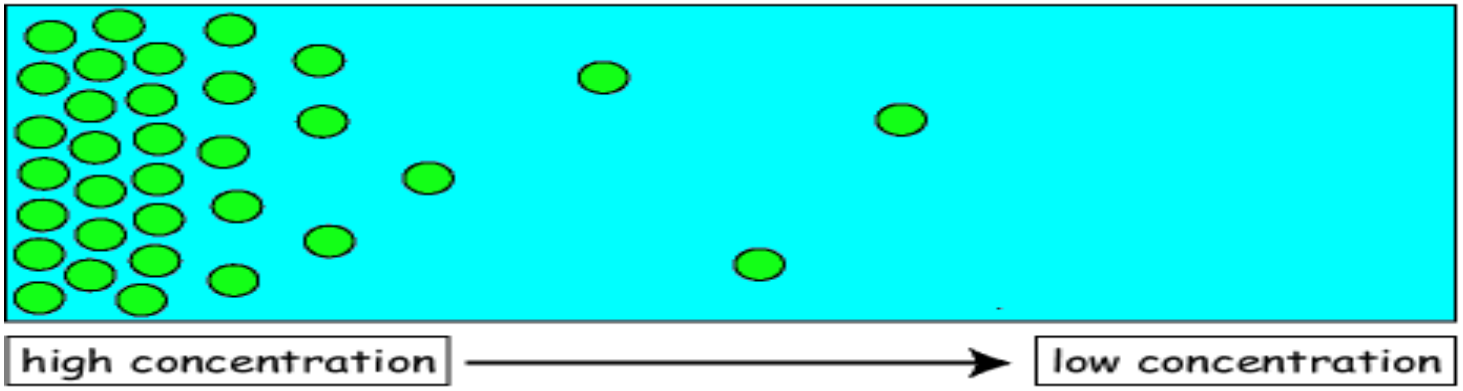
$$\frac{\partial d_{x,y}(t)}{\partial t} = D\nabla^2 d_{x,y}(t)$$

- Physical diffusion is a natural process to diffuse material from high density to low density while maintaining the relative order

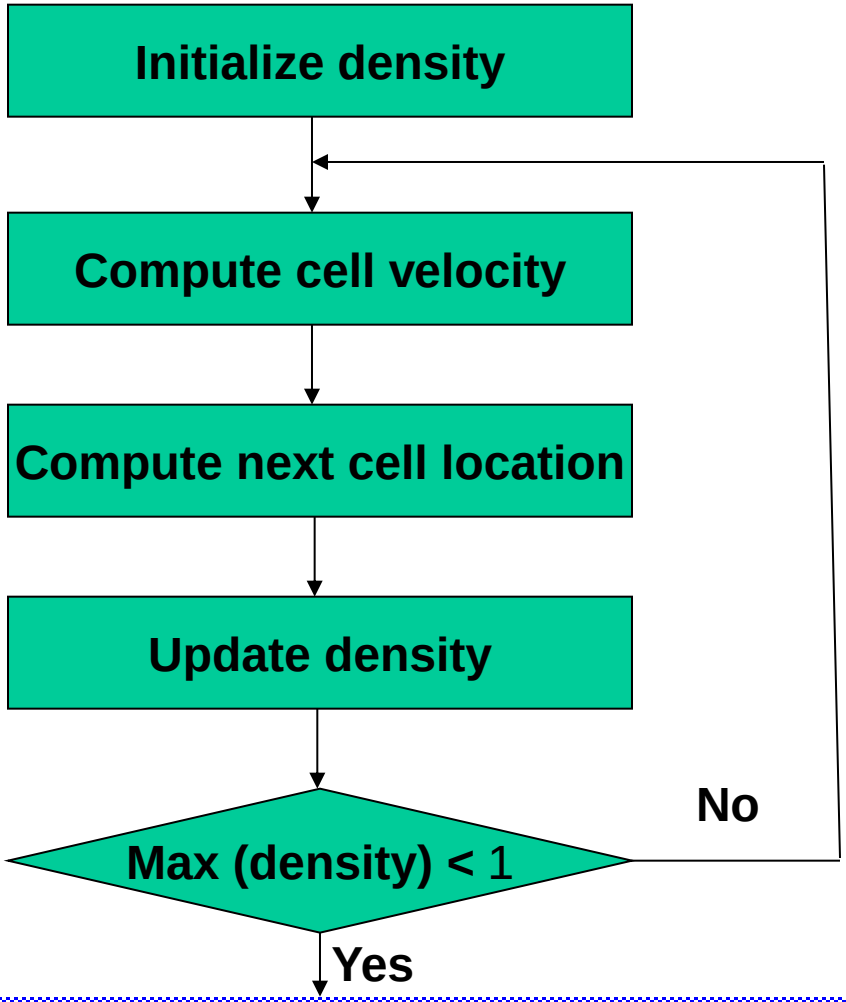


Diffusion Flow

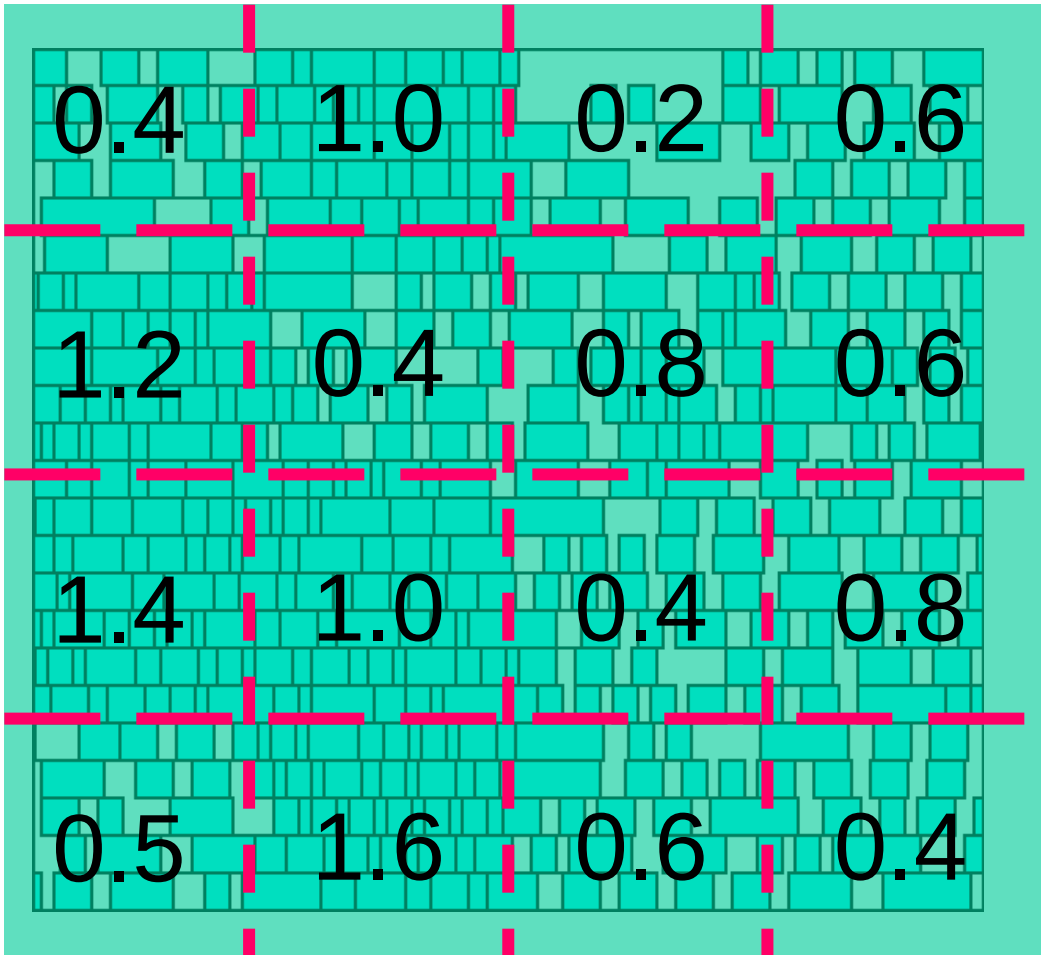
- The velocity of the diffusion flow is proportional to the local density gradient $V = -\frac{\nabla d}{d}$
- Its final location can be computed by the velocity integral $X = \int V dt$



Diffusion-Based Placement

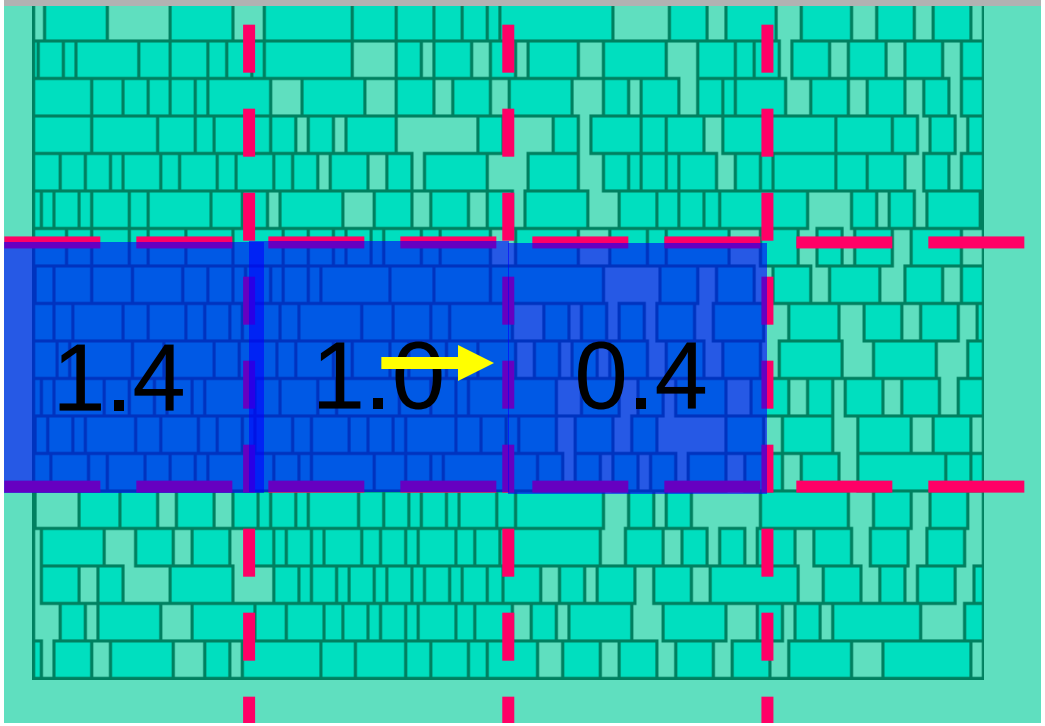


Initialize Bin Density



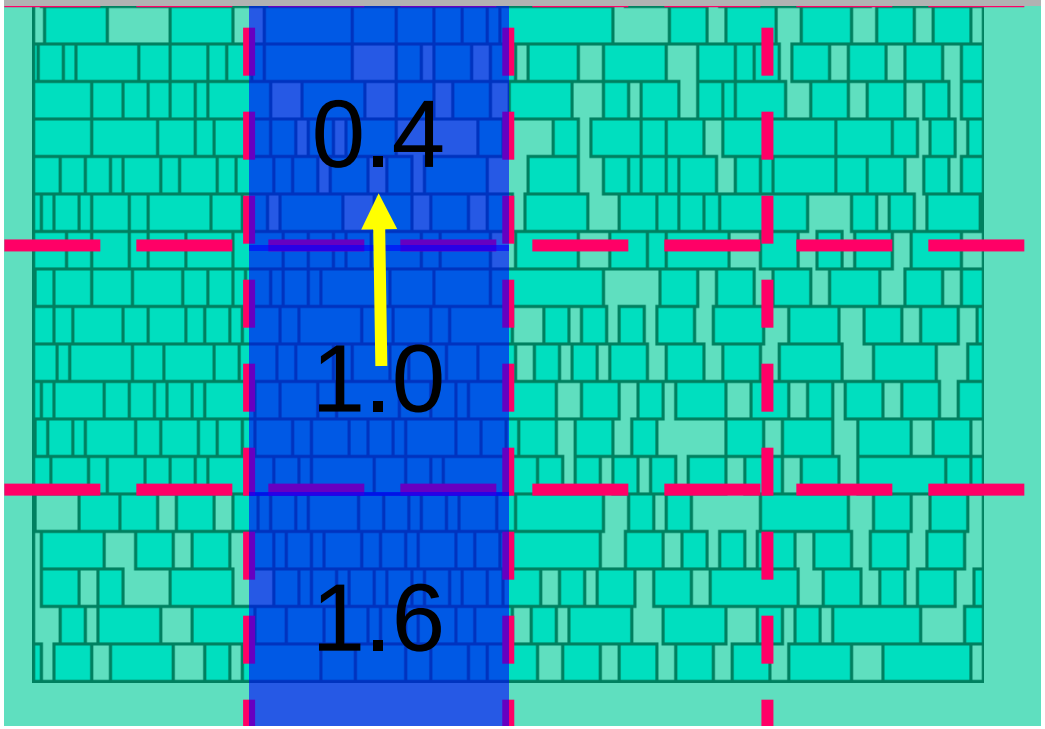
Compute Horizontal Bin Velocity

$$v_{1,1}^{H} = -\frac{0.4 - 1.4}{2 \times 1.0} = 0.5$$

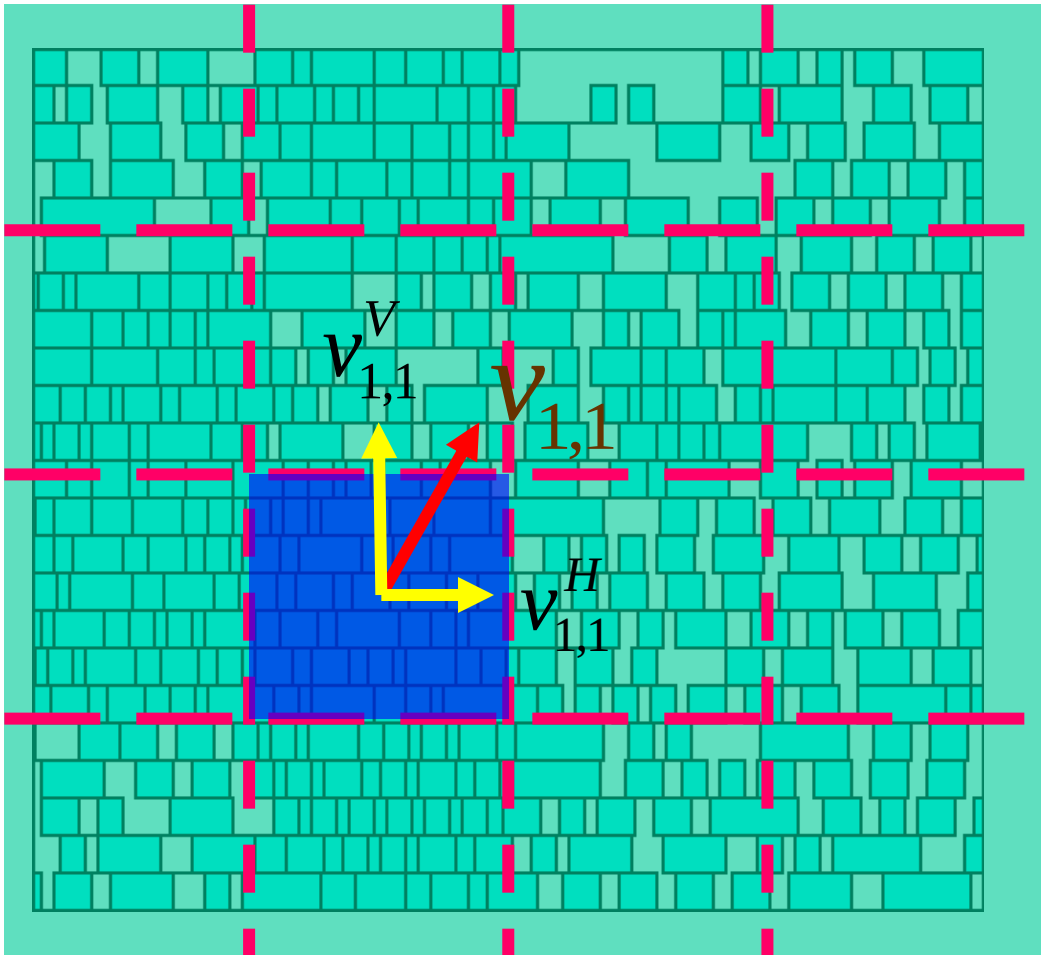


Compute Vertical Bin Velocity

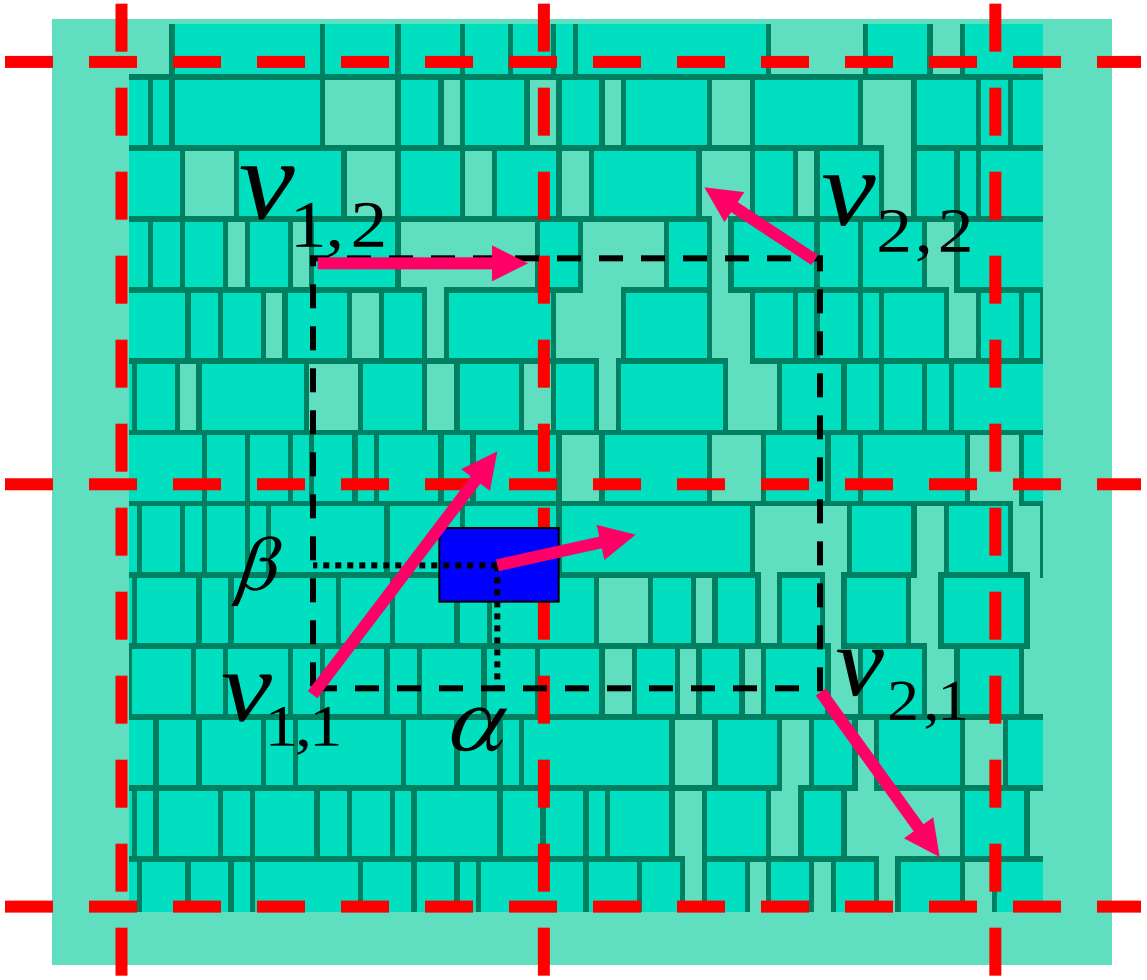
$$v_{1,1}^{V} = -\frac{0.4 - 1.6}{2 \times 1.0} = 0.6$$



Compute Bin Velocity



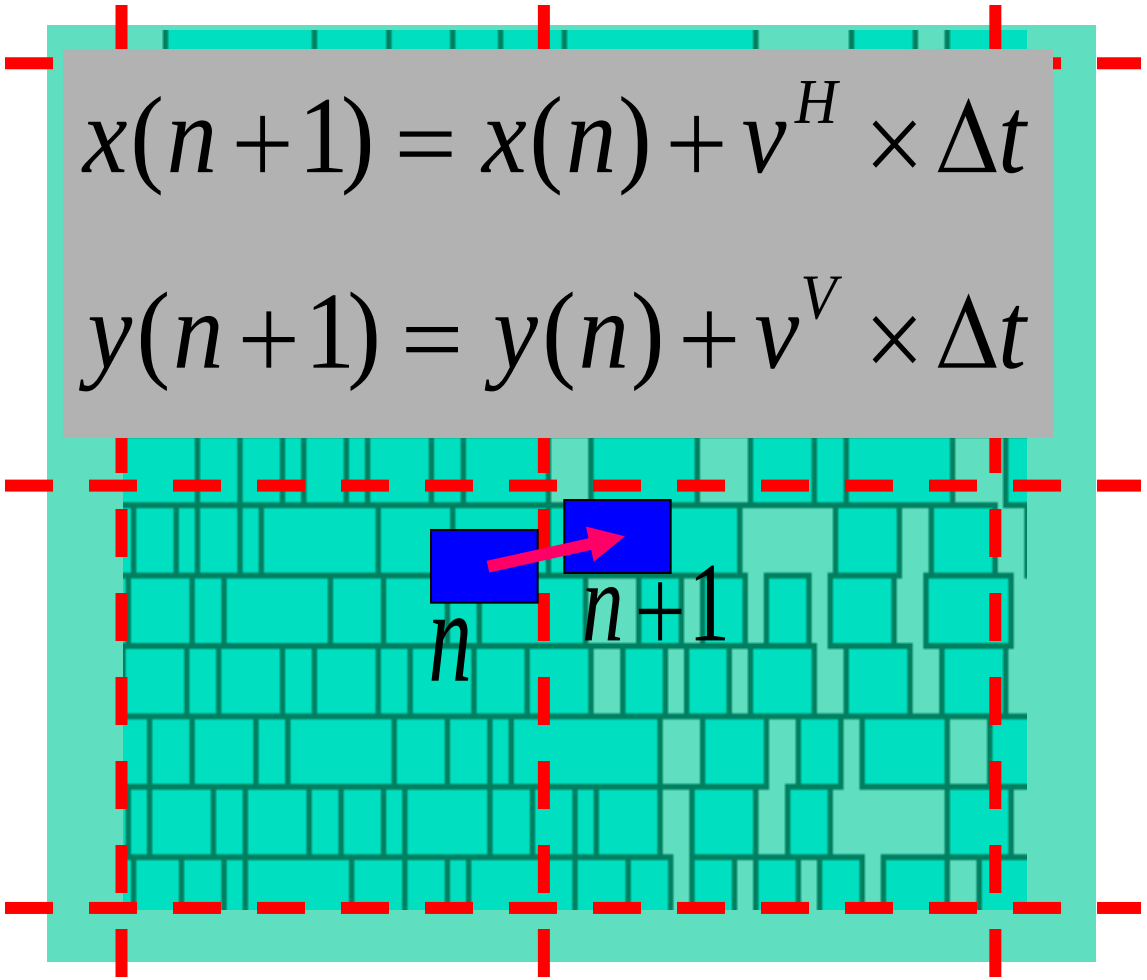
Velocity Interpolation



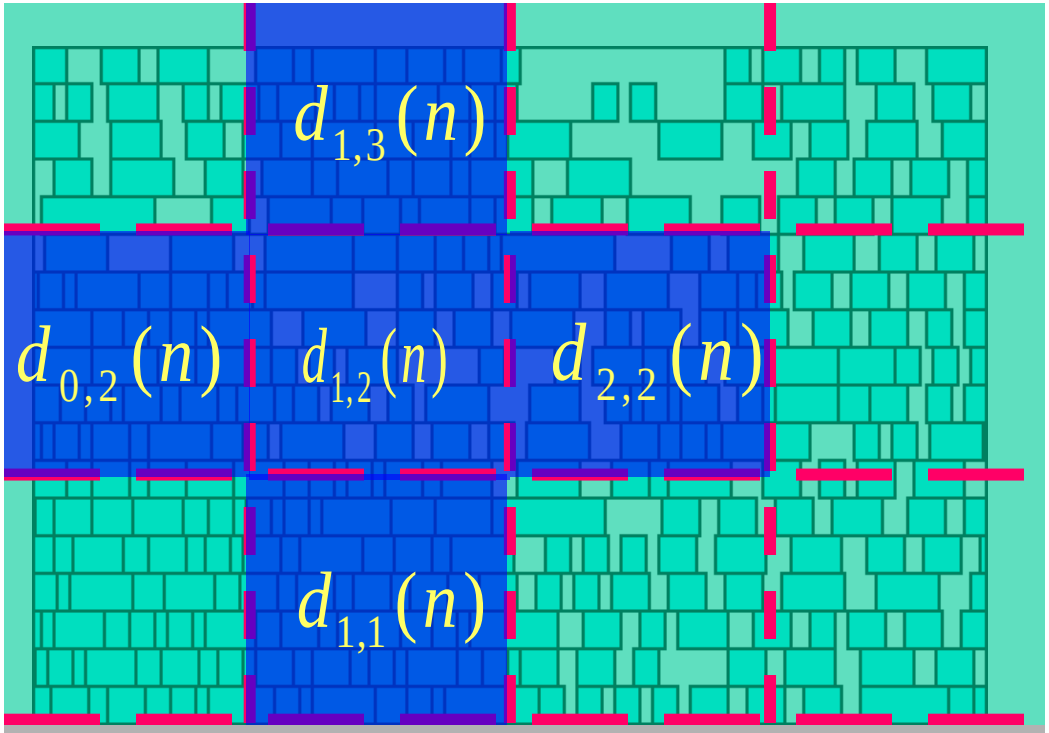
Move Cell

$$x(n+1) = x(n) + v^H \times \Delta t$$

$$y(n+1) = y(n) + v^V \times \Delta t$$

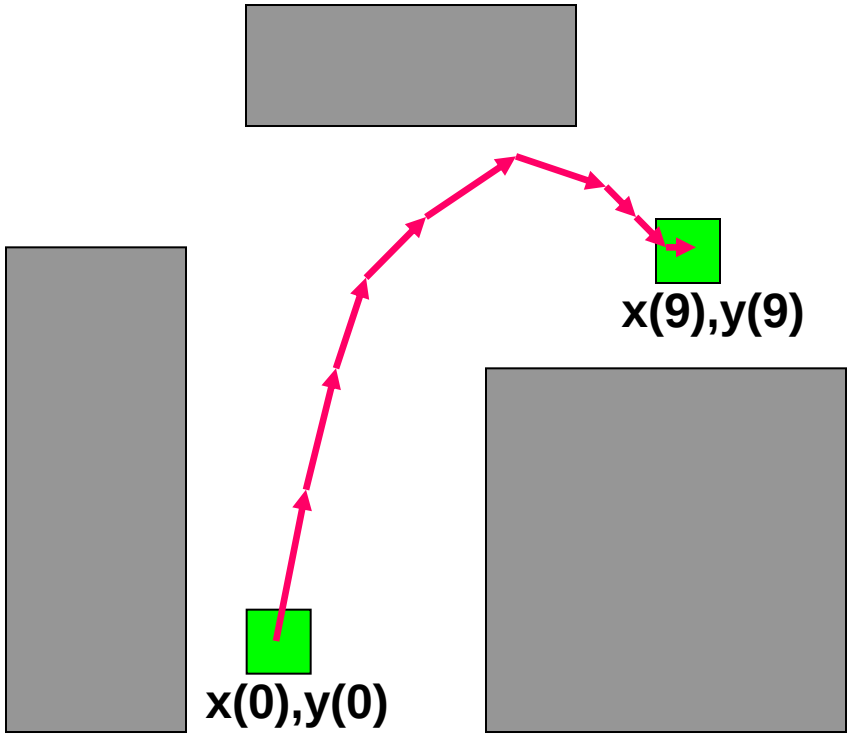


Update Bin Density



$$d_{1,2}(n+1) = d_{1,2}(n) + \frac{\Delta t}{2}[d_{0,2}(n) + d_{2,2}(n) - 2d_{1,2}(n)]$$
$$+ \frac{\Delta t}{2}[d_{1,1}(n) + d_{1,3}(n) - 2d_{1,2}(n)]$$

Cell Movement Example

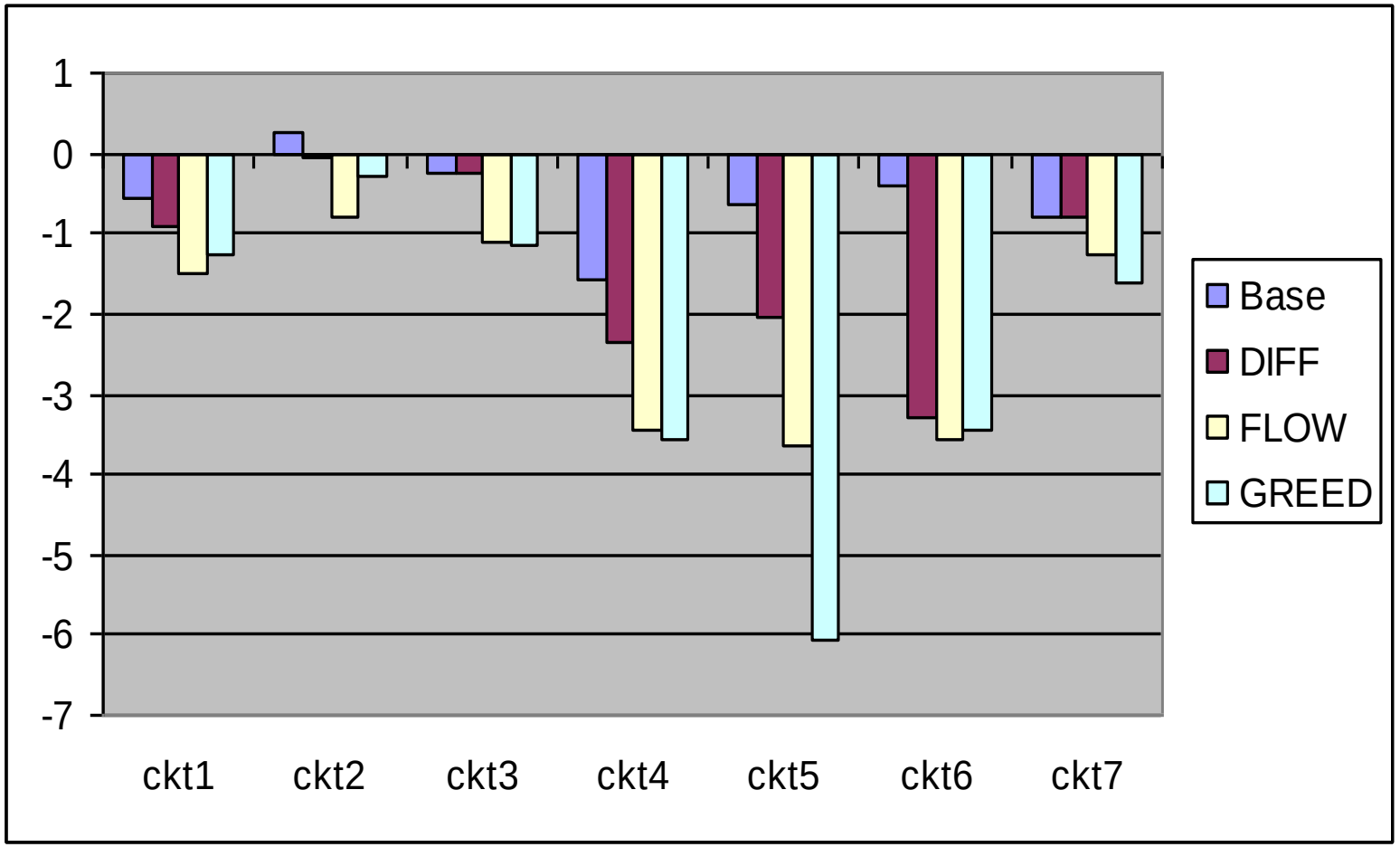


Experiments

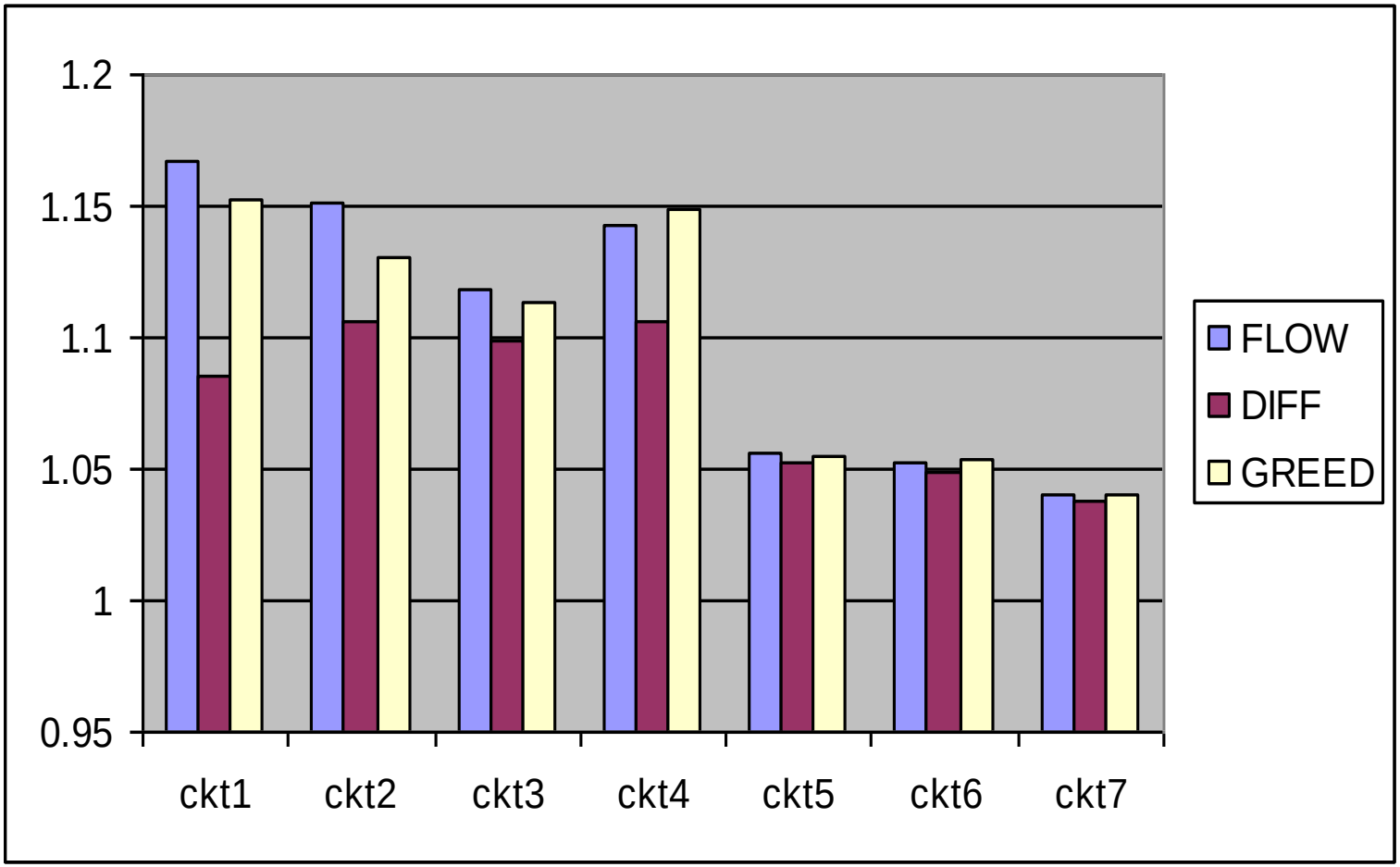
- Compare three legalization algorithms
  - GREED: Greedy algorithm that places cells at nearest possible location via slide and spiral search (packing)
  - FLOW: Network flow based algorithm
  - DIFF: Diffusion based legalization algorithm
- Test on seven industry testcases (Run IBM Placement Driven Synthesis (PDS) with non-overlap mode, then use cell expansion to generate illegal placement)

testcases	# of cells	size(mm)	inflation(%)
ckt1	64K	1.9x1.9	23.1
ckt2	72K	2.3x2.3	32.4
ckt3	159K	5.3x5.3	47.2
ckt4	216K	9.0x9.0	40.4
ckt5	307K	11.9x11.9	25.4
ckt6	440K	10.0x10.0	42.2
ckt7	1076K	13.0x13.0	18.9

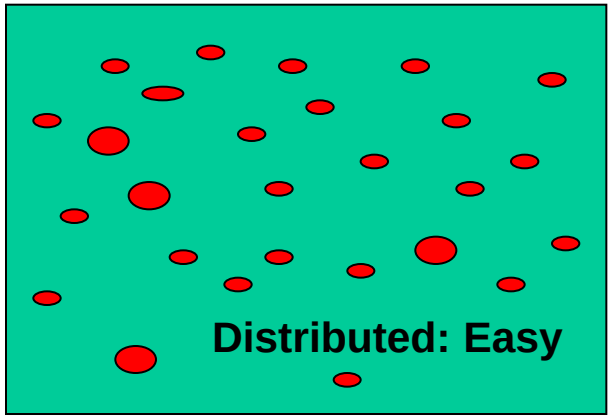
Worst Negative Slack

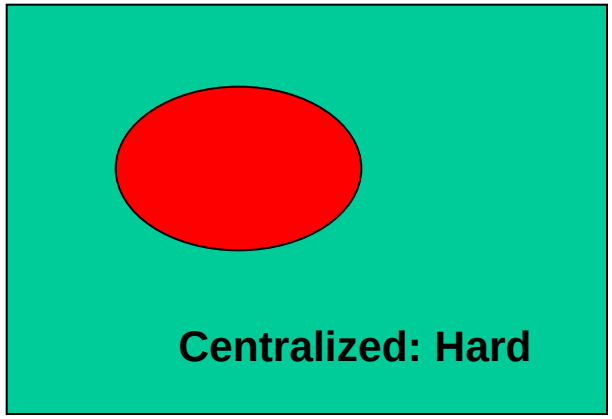


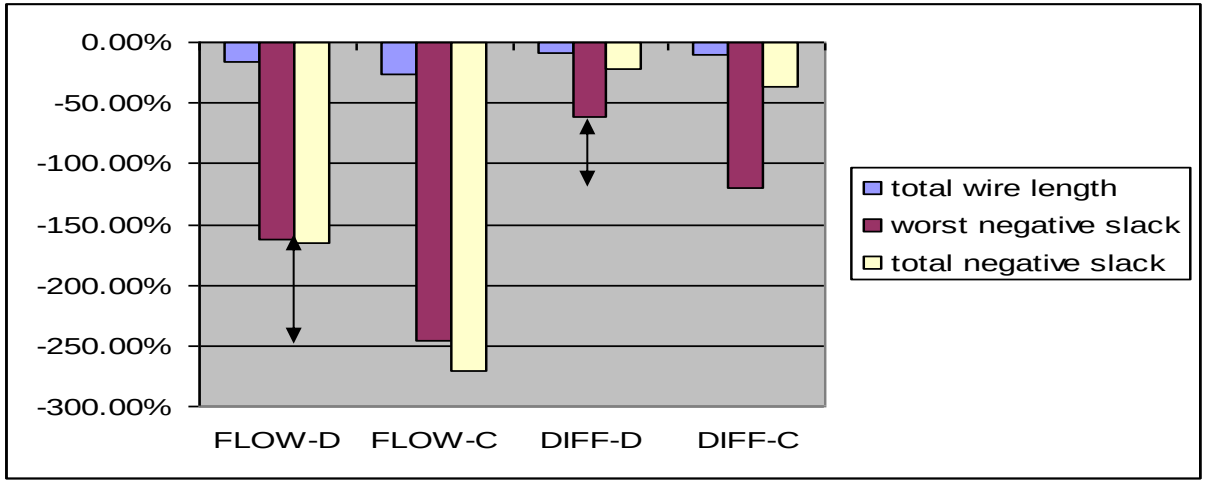
Total Wire Length (scaled)



Overlap Distribution Effects







QoR Degradation of Distributed vs. Centralized

Computational Geometry Based Placement [Luo+, ICCAD'05]

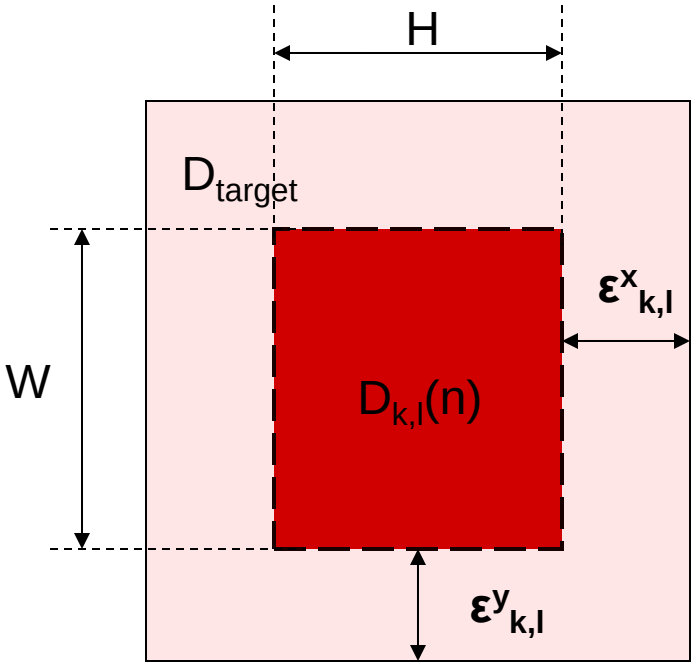
- Another view for stable placement migration to maintain the "relative" geometry order
  - Bin-based Spreading
    - Iterative bin stretching
    - Cell interpolation
  - Delaunay triangulation based legalization
    - Delaunay triangulation of the placement region
    - Fine-grain density distribution and overlapping removing
  - Geometric placement stability metrics

Bin Stretching

- Compute the bin density *Dk,l(n)* in iteration *n*
- Compute $\varepsilon^x_{k,l}$ and $\varepsilon^x_{k,l}$, the stretching/shrinking amount in horizontal and vertical directions

$$\varepsilon^x_{k,l} = (\sqrt{\frac{D_{k,l}(n)}{D_{target}}} - 1)W$$

$$\varepsilon^y_{k,l} = (\sqrt{\frac{D_{k,l}(n)}{D_{target}}} - 1)H$$



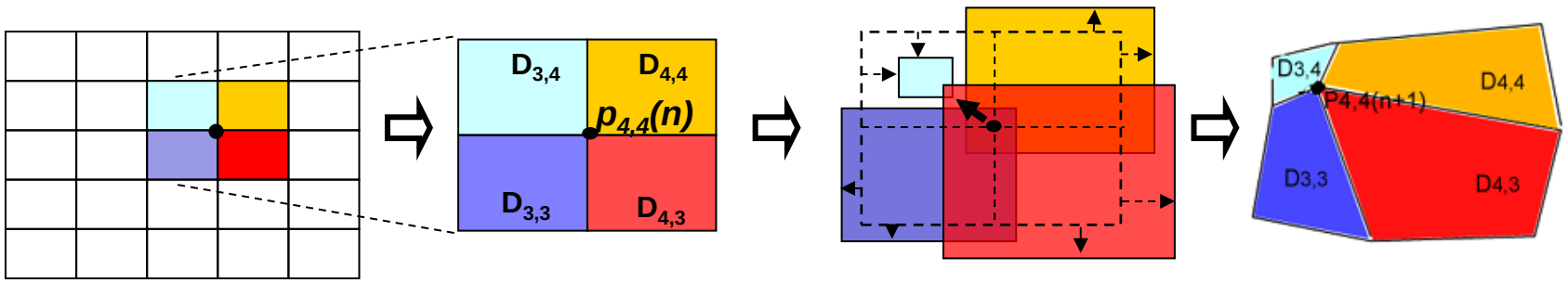
Bin Stretching → Corner Stretching

- Stretch each bin individually will generate bin overlaps
  - stretch bin corners $p^x_{k,l}$, $p^y_{k,l}$ instead

$$p^x_{k,l}(n+1) = p^x_{k,l}(n) + 0.5\delta(\varepsilon^x_{k-1,l-1} + \varepsilon^x_{k-1,l} - \varepsilon^x_{k,l-1} - \varepsilon^x_{k,l})$$

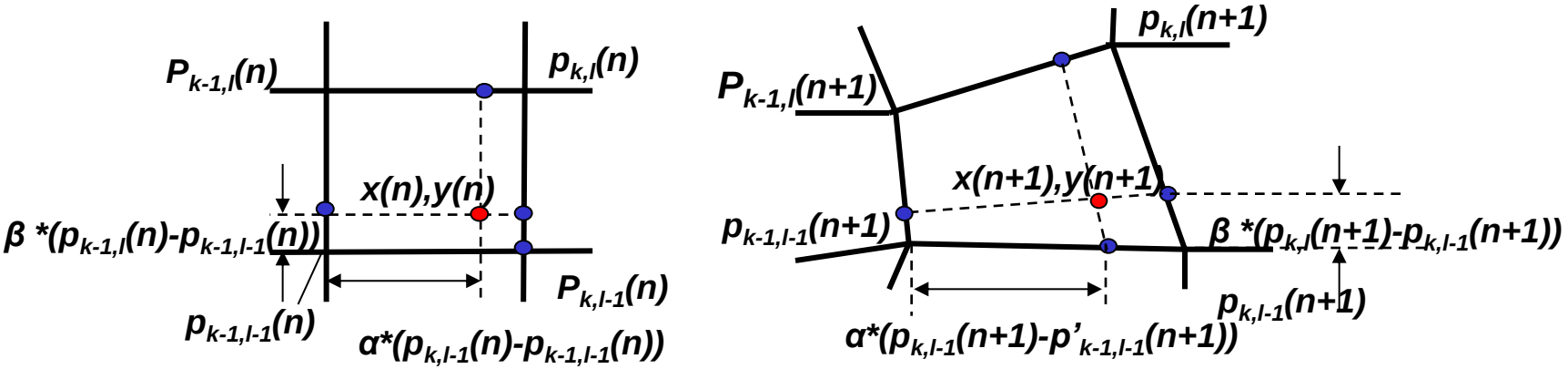
$$p^y_{k,l}(n+1) = p^y_{k,l}(n) + 0.5\delta(\varepsilon^y_{k-1,l-1} + \varepsilon^y_{k-1,l} - \varepsilon^y_{k,l-1} - \varepsilon^y_{k,l})$$

  - If a corner is on a fixed boundary, take the 0.5 factor off
  - δ is for controlling the smoothness of spreading process



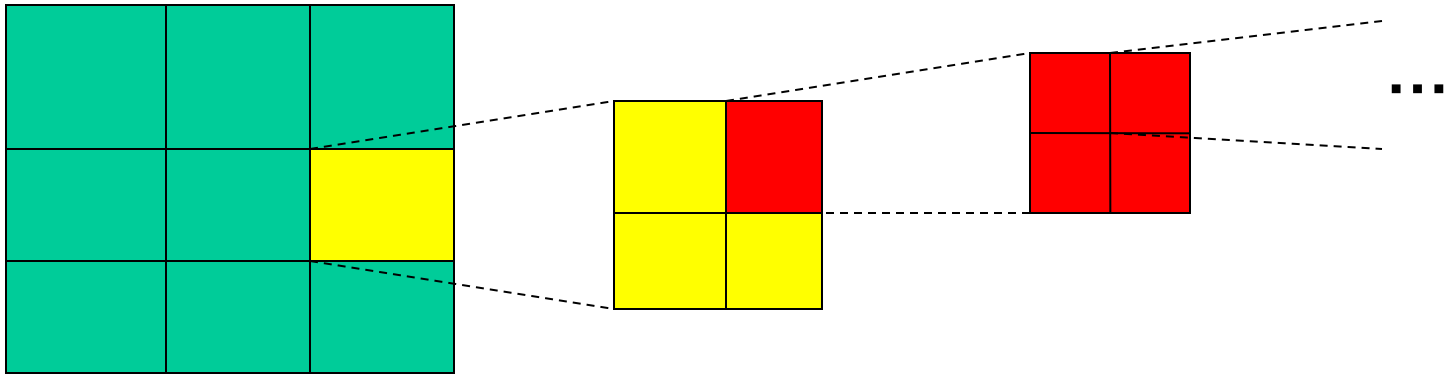
Cell Location Interpolation

- Linearly interpolate cell coordinates *x(n+1)*, *y(n+1)* from *x(n)*, *y(n)* inside the new stretched bin



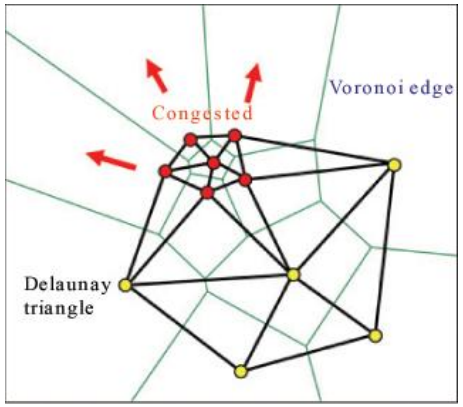
Hierarchical Iterative Spreading

- In every iteration
  - The bin boundary is restored after cells are moved to new locations
  - Re-compute the bin density and repeat previous steps
  - Stops once all bin densities are under D_{target}
- Multilevel spread
  - Use large bin in the beginning
  - Divide large bin into smaller bins, selectively do *regional spreading*

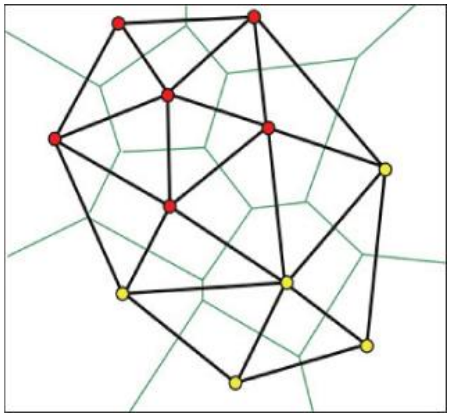


Delaunay Triangulation

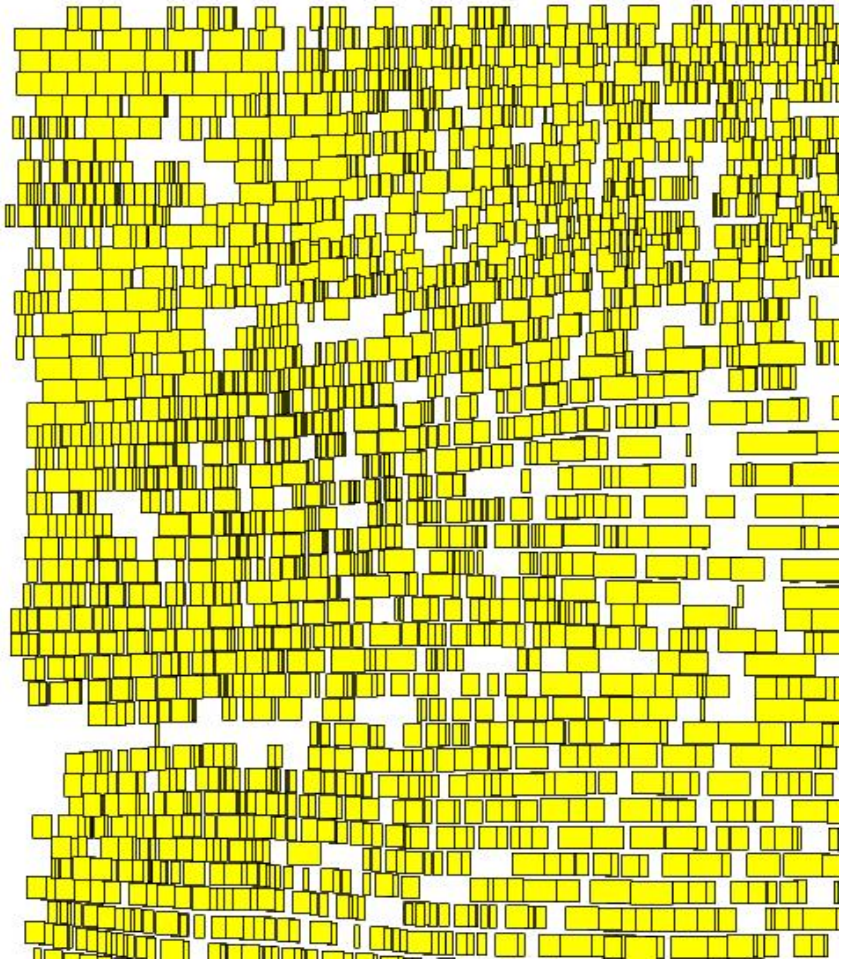
- What is Delaunay triangulation?
  - The dual of the *Voronoi diagram*
  - Given a set of point set *S*, the Voronoi diagram of *S* is the partition of space such that each point has a closet subspace.

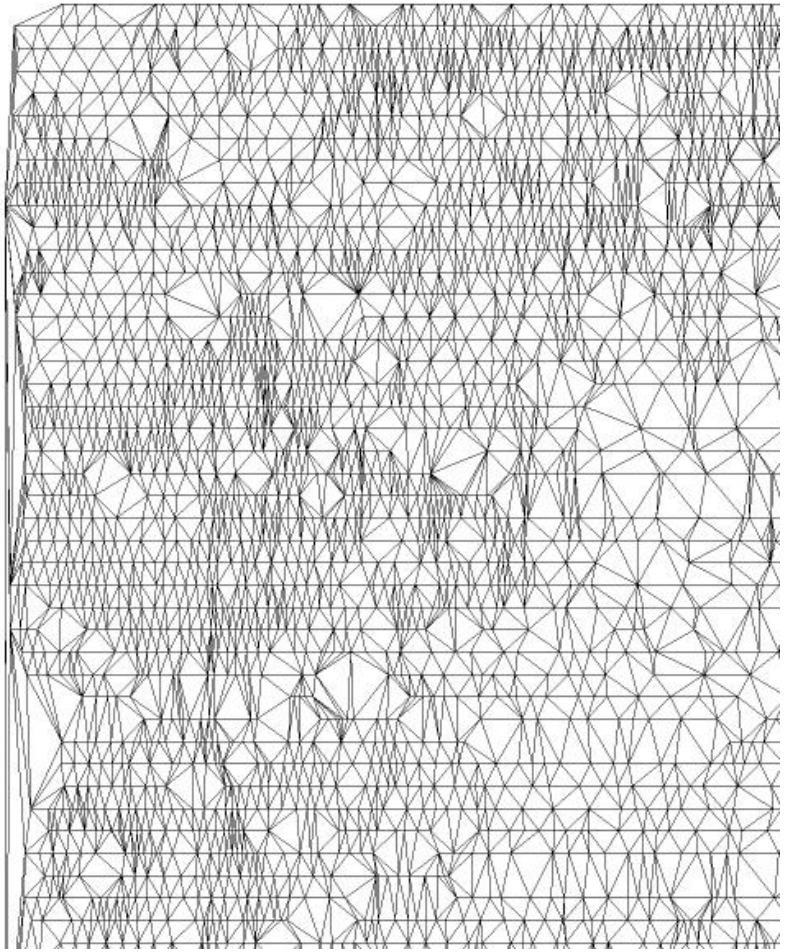


- Delaunay triangulation captures the geometric ordering
  - Any point is connected with all nearest neighbors by Delaunay edges



Delaunay Triangulate a Placement





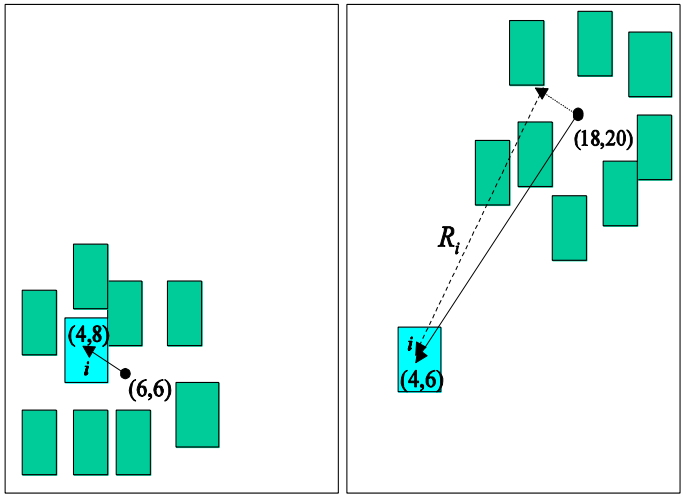
Delaunay Triangulation Based Legalizer

- Delaunay-triangulate the placement region
- Delaunay-based cell spreading
- Snap cells to closest rows
- Row balancing (multiple iterations)
  - Triangulate the region
- Final legalization (multiple iterations)

Geometric Placement Stability Metrics

- Previous metric: absolute cell movement and net cluster metrics, [Alpert et al, ASPDAC'04]
  - Good for capturing absolute cell movements for entire chip
  - Problem: relative movement is important
- Proposed metric: relative cell movement metric
  - Measure cell movement relative to nearest neighbors
  - Capture significant cell movements

$$x_i^{GC} = \frac{max(x_j) + min(x_j)}{2}$$
$$y_i^{GC} = \frac{max(y_j) + min(y_j)}{2}$$
$$\hat{x}_i^{GC} = \frac{max(\hat{x}_j) + min(\hat{x}_j)}{2}$$
$$\hat{y}_i^{GC} = \frac{max(\hat{y}_j) + min(\hat{y}_j)}{2}$$
$$R_i = [(\hat{x}_i - \hat{x}_i^{GC}) - (x_i - x_i^{GC}]^2 + [(\hat{y}_i - \hat{y}_i^{GC}) - (y_i - y_i^{GC})]^2$$
$$S_G = \frac{\sum_{j \in \mathbf{c}_{1\%}} R_j}{N}$$



Experimental Results

- CGL: **C**omputational **G**eometry **L**egalization

Table 1: TWL Comparison (scaled)

testcases	*GREEDY*	*FLOW*	*CGL*
ibm01	1.282	1.314	1.191
ibm02	1.056	1.064	1.013
ibm03	1.101	1.096	1.061
ibm04	1.165	1.195	1.100
ibm05	1.016	1.018	1.014
ibm06	1.111	1.123	1.035
ibm07	1.141	1.139	1.039
ibm08	1.154	1.153	1.042
ibm09	1.211	1.221	1.069
ibm10	1.180	1.181	1.030
ibm11	1.193	1.187	1.056
ibm12	1.162	1.167	1.050
ibm13	1.229	1.233	1.059
ibm14	1.239	1.235	1.047
ibm15	1.274	1.273	1.065
ibm16	1.313	1.318	1.040
ibm17	1.267	1.285	1.040
ibm18	1.296	1.318	1.054
Average	1.188	1.197	1.056

Table 2: S_G Comparison

testcases	*GREEDY*	*FLOW*	CGL_B	*CGL*
ibm01	52199	37785	12685	11349
ibm02	44735	24126	5225	5172
ibm03	58375	52356	41099	26081
ibm04	119647	58967	52199	39039
ibm05	55040	65344	53387	49538
ibm06	60242	48890	4555	4750
ibm07	131024	119809	5232	5764
ibm08	153395	119758	3610	3750
ibm09	168791	154334	20743	16126
ibm10	302222	238185	5057	4740
ibm11	224925	190321	32519	50898
ibm12	361702	331289	5270	5088
ibm13	370440	349016	177377	117833
ibm14	557898	456627	4581	4364
ibm15	698634	746372	130784	123811
ibm16	937453	746372	4435	4129
ibm17	1169281	1087672	14648	11019
ibm18	1116867	985042	3224	3267
Average	365715	334406	31855	26858

Experiments: CPU time

- CGL are efficient

Table 3: CPU Time Comparison (s)

testcases	*GREEDY*	*FLOW*	*CGL*
ibm01	11	10	10
ibm02	23	25	23
ibm03	28	26	25
ibm04	62	29	30
ibm05	33	37	40
ibm06	69	53	35
ibm07	162	150	55
ibm08	307	242	65
ibm09	325	289	75
ibm10	806	575	89
ibm11	617	490	95
ibm12	1299	807	121
ibm13	939	734	134
ibm14	3654	2240	231
ibm15	5210	3643	430
ibm16	9927	6280	511
ibm17	11363	8280	558
ibm18	12724	9160	661

